Network Working Group
Internet-Draft
Intended status: Experimental
Expires: January 2027

I. Paruk
Envoir
July 2026

Decentralized Message Transfer & Access Protocol (DMTAP)

`draft-paruk-dmtap-00`

Abstract

DMTAP is an open protocol for sovereign, metadata-private communication — mail, chat, and files — and decentralized identity/login, over a peer-to-peer mesh, with an optional bridge to legacy email. It unifies message *transfer* (the role of SMTP) and message *access* (the role of IMAP) into one decentralized, encrypted protocol, and extends the same keypair identity into decentralized web login. A user's identity is a keypair; a human name (an 8-word key-derived name, an @handle, or a domain) is a replaceable pointer to it. Content, authenticity, and the social graph are protected: messages are end-to-end encrypted and signed, and who-talks-to-whom is hidden from a global passive observer. DMTAP composes existing standards (MLS, HPKE, JMAP, libp2p, DNS, key transparency, Privacy Pass, WebAuthn, OAuth, DID); the novelty is the composition and transport, not new cryptography.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of an open, royalty-free specification. Internet-Drafts are working documents; this document is a Experimental proposal and is *not* an RFC — an RFC number is assigned only upon publication by the relevant standards body. It is valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use this document as reference material or to cite it other than as "work in progress." This draft will expire on January 2027.

Requirements Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

Copyright Notice

Reference implementation: *Envoir* (MIT). Message object: *MOTE*.

Table of Contents

0. Overview & Architecture **6**
0.1 Goals . . . 6
0.2 The two components . . . 6
0.3 The three layers of indirection . . . 7
0.4 Message-flow summary . . . 7
0.5 Where state lives . . . 8
0.6 Privacy posture (summary; full model in §6) . . . 8
0.7 Non-goals . . . 8

1. Identity Lifecycle **8**
1.1 Algorithm suites & crypto-agility . . . 9
1.2 Keys and the identity hierarchy . . . 9
1.3 The `Identity` object (published) . . . 10
1.4 Recovery policy (and rotating the recovery methods) . . . 10
1.5 Key rotation . . . 12
1.6 Identity (name) migration — never losing your address . . . 12
1.7 Object summary . . . 13

2. The MOTE Object **13**
2.1 Layered structure . . . 13
2.2 Envelope (CBOR) . . . 13
2.3 Message kinds . . . 14
2.4 Payload (CBOR, encrypted) . . . 15
2.5 Attachments and files . . . 15
2.6 Delivery semantics . . . 16
2.7 Validation (recipient MUST) . . . 16
2.8 Why this shape . . . 17

3. Naming & Directory (name → key) **17**
3.1 Roles (who holds what) . . . 18
3.2 DNS records . . . 18
3.3 Resolution . . . 18
3.4 Trust on first use (TOFU) + pinning . . . 18
3.5 Key transparency (KT) . . . 19
3.6 Optional: self-sovereign naming (un-loseable addresses) . . . 20
3.7 Private lookups . . . 20
3.8 Onboarding tiers (day-one setup) . . . 20
3.9 Names (one identity, many pointers) . . . 21

4. Transport: Mesh, Mixnet, Delivery **23**
4.1 Substrate: libp2p . . . 23
4.2 The `key → location` record (DHT) . . . 23
4.3 Reachability ladder . . . 24
4.4 The mixnet (metadata privacy) . . . 24
4.5 Bulk / file transfer . . . 24
4.6 Privacy tiers . . . 24
4.7 Delivery state machine (sender) . . . 25
4.8 Local, isolated, and delay-tolerant networks . . . 25

5. Messaging & Files (the unified substrate) 25
5.1 Why MLS everywhere . . . 25
5.2 Forward secrecy & (non-)deniability . . . 27
5.3 Async session initiation (MLS-native) . . . 27
5.4 Message kinds in context . . . 27
5.5 Files: content-addressed, chunked, any size . . . 27
5.6 Multi-device (the personal cluster) . . . 28
5.7 Three products, shared components . . . 28
5.8 Groups as addressable identities . . . 28

6. Privacy & Threat Model 30
6.1 Adversary model . . . 30
6.2 What is protected, and how . . . 30
6.3 Mixnet, cover traffic, padding . . . 30
6.4 Why we avoid PIR (honest framing) . . . 31
6.5 Privacy tiers (and where each product sits) . . . 31
6.6 Honest boundaries (what we do NOT claim) . . . 32
6.7 Data at rest . . . 32

7. The Legacy Gateway (optional) 33
7.1 Responsibilities . . . 33
7.2 Inbound . . . 33
7.3 Outbound & DKIM delegation . . . 33
7.4 Statelessness & durability . . . 34
7.5 Decentralization & economics (summary; anti-abuse in §9) . . . 34
7.6 Dual-stack addressing . . . 34
7.7 Fairness, self-host backstop & non-lock-in . . . 34

8. Client Access 35
8.1 JMAP (native) . . . 35
8.2 IMAP / POP / SMTP-submission (compatibility) . . . 35
8.3 Multi-device . . . 36
8.4 Calendar & contacts (first-class, decentralized) . . . 36
8.5 The decentralization invariant (all data classes) . . . 36

9. Anti-Abuse & Postage 36
9.1 Principles . . . 36
9.2 Recipient policy . . . 37
9.3 Anonymous rate-limit tokens (the core mechanism) . . . 37
9.4 Proof-of-work (fallback) . . . 38
9.5 Postage (economic tier + gateway funding) . . . 38
9.6 Gateway-operator accountability . . . 38
9.7 Vouch / introduction (optional) . . . 39
9.8 Mixnet abuse . . . 39

10. Versioning, Conformance, Governance 39
10.1 Protocol versioning . . . 39
10.2 Capability negotiation (dual-stack) . . . 39
10.3 Conformance levels . . . 39
10.4 The spec is authoritative . . . 40
10.5 Governance & licensing (intent) . . . 40
10.6 Roadmap markers (non-normative) . . . 40

11. Grounding & References **40**
11.1 Verified building blocks . . . 40
11.2 Corrections applied after verification . . . 41
11.3 Consolidated honest limits . . . 42
11.4 Implementation-stack notes (reference, Rust) . . . 42
11.5 Selected bibliography . . . 42

12. Operators, the Seam & Sustainability **42**
12.1 Two deployment modes . . . 43
12.2 The operator seam . . . 43
12.3 The inviolable rule (normative) . . . 43
12.4 Business & licensing model (informative) . . . 43
12.5 Honest limits . . . 44

13. DMTAP-Auth — Decentralized Identity & Login **44**
13.1 Goals & non-goals . . . 44
13.2 Substrate reuse . . . 45
13.3 The native login ceremony . . . 45
13.4 Sessions: key-bound, not bearer . . . 46
13.5 Capabilities & delegation . . . 46
13.6 Legacy bridge: OIDC / OAuth compatibility . . . 46
13.7 Grounding & honest limits . . . 47

14. Scaling & Deployment **48**
14.1 Node classes . . . 48
14.2 Horizontally-scalable gateways (normative patterns) . . . 48
14.3 The mobile-only user (no always-on box) . . . 49
14.4 The always-on-box user . . . 49
14.5 Relay & buffer scaling . . . 49
14.6 Hosted multi-tenant topology (the operator) . . . 50
14.7 Grounding . . . 50

15. Normative & Informative References **50**
15.1 Normative — cryptography & encoding . . . 50
15.2 Normative — transport, mesh & naming . . . 51
15.3 Normative — legacy mail interop (the gateway & client edges) . . . 51
15.4 Normative — identity, auth & anti-abuse . . . 51
15.5 Informative — precedent & rationale . . . 52
15.6 On reproduction . . . 52

16. Numeric Parameters (v0) **52**
16.1 Time, replay & clocks . . . 52
16.2 Naming, KT & DHT . . . 53
16.3 Mixnet & privacy . . . 53
16.4 File tiers & transfer . . . 53
16.5 Anti-abuse . . . 53
16.6 Transport & relay . . . 54
16.7 Crypto suites . . . 54

0. Overview & Architecture

0.1 Goals

DMTAP is a protocol for authenticated, encrypted, metadata-private messaging between self-sovereign identities, with async (store-and-forward) delivery, no required central party, and an optional bridge to legacy email. One substrate carries **mail, chat, files**, and **decentralized identity/login** — the same keypair that receives your mail logs you in across the web (§13), with no central identity provider.

Concretely, DMTAP MUST provide:

1. **Sovereign identity** — a keypair you own; no account with any provider is required to *be* a DMTAP identity. The same identity serves mail, messaging, files, and web login (§13).
2. **Reachability without a static IP** — a node behind CGNAT, on a dynamic IP, is reachable by its key.
3. **Content, authenticity, and metadata privacy** — messages are end-to-end encrypted and signed, and the social graph (who talks to whom, when, how much) is hidden from a global passive observer.
4. **Continuity** — you never lose access to your identity (redundant, rotatable recovery) and can migrate your human name without losing existing contacts.
5. **Legacy interoperability** — you can exchange mail with the existing SMTP world, and existing OIDC apps can consume DMTAP login through a bridge (§13.6).
6. **Scales across device classes** — always-on nodes (Pi/NAS/VPS) hold the mailbox; intermittent devices (laptops, phones) participate as thin clients or push-woken nodes; gateways scale horizontally (§14).
7. **Future-proofing** — crypto-agility, transport independence, standards reuse; the system gets *simpler* as IPv6 spreads and legacy fades.

0.2 The two components

DMTAP is, deliberately, only two pieces of software plus DNS (which we do not build):

The Node (`node/`)

One binary, installed on any box that runs most of the time. It holds **all durable state** and does **all the real work**:

- Identity: the root keypair, device subkeys, recovery policy (§1).
- Store: the mailbox and file blobs (encrypted MOTEs + content-addressed chunks) (§2, §5).
- Mesh participation: peer discovery (DHT), relaying for others, delivery (§4).
- Mixnet client: onion-wrapping, cover traffic, sealed sender (§4, §6).
- Messaging: MLS groups for 1:1, chat, and file folders; MLS KeyPackages (§5.3).
- Client access: JMAP native, plus IMAP/POP/SMTP-submission compatibility (§8).
- The outbound **retry queue** — durability lives here, not in the middle.

A node MAY additionally run in **relay mode** (help NAT'd peers, if it has a public address) or **mix mode** (be a mixnet hop). These are capabilities of the same binary, not separate programs.

The Gateway (`gateway/`) — optional

The **only** component that speaks SMTP and the **only** one that is not content-blind (the legacy leg is unavoidably plaintext). It:

- receives inbound legacy mail (acts as MX), wraps it into a MOTE, attests it, and delivers into the mesh; returns SMTP `4xx` if the recipient is offline so the *sending* server retries;
- sends outbound legacy mail, DKIM-signing as the user's domain via delegated selectors;
- carries the operational weight the system cannot avoid: **IP reputation**.

A node without legacy correspondents never invokes a gateway. At full adoption, the gateway is unnecessary. The gateway MAY be the node binary run in `--gateway` mode by an operator with a reputable IP and a domain.

DNS (not built here)

The naming substrate that maps a human name to a key. We publish and read records; we do not run DNS. See §3. DNS holds the **stable** binding (name → key); the mesh holds the **dynamic** binding (key → current location).

0.3 The three layers of indirection

The core trick that frees an address from any IP:

```
abc@def.com  ──DNS/§3──▶  public key   ──mesh/§4──▶  current location
   NAME                    IDENTITY                    (IP/relay/mix path)
   (human, stable)         (durable, portable)         (ephemeral, self-updated)
```

- **Name → key** is stable and lives in DNS (+ key transparency). It changes only when you migrate names.
- **Key → location** is dynamic and lives in the mesh (a signed, TTL'd DHT record the node republishes as its address changes).
- The **key is the identity.** Existing contacts route by key via the mesh and never need DNS again after first contact; a lost domain is a change of *name*, not of identity (§1.6).

0.4 Message-flow summary

DMTAP → DMTAP (the common path)

```
1. resolve  abc@def.com → recipient key K            (§3; cached/pinned after first contact)
2. fetch    K's KeyPackages + current location          (§4 DHT, §5.3 KeyPackages)
3. build    a MOTE: sealed-sender, MLS/HPKE-encrypted to K, signed  (§2, §5, §6)
4. send     through the mixnet (private tier) or direct (fast tier) (§4, §6)
5. recipient node receives, verifies, decrypts, stores; acks        (§2, §4)
   (sender's node retries until ack — durability at the edge)
```

No gateway, no SMTP, no plaintext outside the endpoints.

Legacy → DMTAP (inbound)

```
gmail ──SMTP──▶ Gateway ──wrap+attest+encrypt to K──▶ mesh ──▶ recipient node
                (if node offline: return SMTP 4xx; Gmail retries)
```

DMTAP → Legacy (outbound)

```
node ──MOTE(legacy)──▶ Gateway ──SMTP + delegated DKIM──▶ gmail
                       (node retries on failure)
```

0.5 Where state lives

State	Location	Notes
Keys, mailbox, files, retry queue	**Node** (the edge)	All durable state
Name → key	**DNS** + key-transparency log	Stable; small
Key → location	**Mesh DHT**	Dynamic; signed; TTL'd; self-republished
In-flight ciphertext	**Mixnet / relay**	Held only until delivered; content-blind
Legacy reputation	**Gateway**	The only non-trivial operational cost

The middle (mesh, mixnet, gateway) holds **no durable user data**. Durability is always punted to an edge: the sender's node retries; inbound legacy leans on the sending server's SMTP retry.

0.6 Privacy posture (summary; full model in §6)

- **Content & authenticity:** end-to-end encrypted (MLS/HPKE) and signed. Always.
- **Sender metadata:** hidden via **sealed sender** — intermediaries never learn the sender.
- **Social graph & timing:** hidden via a **mixnet** (onion routing + mixing delays) plus **cover traffic** and **size padding**. Email's asynchrony is what makes full-strength mixing affordable.
- **Recipient retrieval:** the **always-on node receives by push** through the mixnet, so there is *no store-and-poll step to hide* — the hardest metadata problem is dissolved by architecture, not by expensive PIR.
- **Discovery:** name→key lookups are routed *through* the mixnet, so the directory does not learn who is looking up whom.
- **Privacy tiers:** messages may choose `private` (full mixnet, minutes of latency) or `fast` (direct/low-hop, seconds, less metadata protection). Default is `private`; bulk file transfer uses `fast` for the payload and `private` for its control message.

Honest boundary: DMTAP targets a **global passive adversary**. Perfect resistance to a global *active* adversary with unlimited resources is not claimed; see §6.

0.7 Non-goals

- Real-time voice/video (separate WebRTC/SFU architecture).
- Blockchain/consensus (except optional self-sovereign naming in §3).
- Server-side search or server-side spam ML (search is on-device; anti-abuse is §9).

1. Identity Lifecycle

Identity is the foundation everything else hangs off. The governing principle:

> **Your key is your identity. Your domain, your IP, and your provider are all replaceable pointers to it.**

This section defines keys, the key hierarchy, the recovery policy (and how recovery methods are themselves rotated), key rotation, and name/identity migration. All of these are **signed, versioned objects** published to the key-transparency log (§3), so every change is authenticated and every unauthorized change is *detectable*.

1.1 Algorithm suites & crypto-agility

Every signed/encrypted object carries a `suite` identifier. Implementations MUST reject unknown suites (fail closed), never guess.

`suite`	Sign	KEM / PKE	AEAD	Status
`0x01`	Ed25519	X25519 (HPKE)	ChaCha20-Poly1305	v0 REQUIRED
`0x02`	Ed25519 + ML-DSA-65	X25519 + ML-KEM-768 (hybrid)	ChaCha20-Poly1305	RESERVED (P

Suite `0x02` is the post-quantum migration target. A node MAY hold keys in multiple suites during migration; the transparency log records which suite is current. Downgrade is prevented by pinning (§3) and by the transparency log's monotonic history.

1.2 Keys and the identity hierarchy

An identity is rooted in a **root identity key** (`IK`), an offline-capable long-term signing keypair. `IK` signs everything authoritative but is used rarely; it SHOULD be held in cold form / recovery custody (§1.4).

```
IK  (root identity key, Ed25519[+ML-DSA])        ← the identity; rarely used
 │  signs ↓
 ├── DeviceKey_1  (per-device signing subkey)     ← day-to-day signing/auth
 ├── DeviceKey_2
 ├── KeyPackages   (MLS: identity/signature + HPKE-init + PQ-KEM keys)   ← async join (§5.3)
 └── RecoveryPolicy vN  (§1.4)                     ← how the identity is recovered
```

- **Device keys** authorize a specific device (phone, laptop, the always-on box). Each is a signing subkey signed by `IK` (a `DeviceCert`), with a `label` , `created` , and optional `expires` . Devices form the owner's **personal cluster** and share the mailbox via the encrypted CRDT sync of §5.
- **KeyPackages** (§5.3) let others start an encrypted MLS session with the identity while every device is offline. KeyPackages are signed by a device key; one-time KeyPackages are consumed per session.
- A DMTAP **address key** presented to correspondents is `IK` 's public half (the stable identity). Correspondents pin it on first use (§3).

DeviceCert (CBOR)

```
DeviceCert {
  suite:      u8,
  ik:         bytes,        // root identity public key
  device_key: bytes,        // device signing public key
  label:      tstr,         // "phone", "home-box", ...
  created:    u64,          // ms epoch
  expires:    ?u64,
  caps:       [+ tstr],     // "send","recv","relay","mix","gateway","admin"
  sig:        bytes,        // IK over the CBOR-encoded fields above
}
```

`caps` gates what a device *may participate in*. **No single device — including an `admin` device — may unilaterally change the recovery policy.** Changing `RecoveryPolicy` (§1.4) always requires either `IK` directly or satisfaction of the current `rotate_threshold` quorum; an

`admin` device counts only as *one factor* toward that quorum (it may, e.g., be a required member of the quorum, but never sufficient alone). This prevents a single stolen `admin` device from locking the owner out by rewriting recovery. See §1.4 for the authoritative signer rule.

1.3 The `Identity` object (published)

The current public identity is a signed, versioned object; its hash is the anchor everyone pins.

```
Identity {
  suites:   [+ u8],         // algorithm suites this identity supports, preference-ordered
  iks:      { u8 => bytes },// identity public key per suite (Ed25519 for 0x01, +ML-DSA for 0x02)
  version:  u64,            // monotonically increasing
  devices:  [* DeviceCert],
  keypkgs:  KeyPackageBundleRef,  // location + hash of the current KeyPackage bundle (§5.3)
  recovery: RecoveryPolicyRef,   // hash of the current RecoveryPolicy (§1.4)
  names:    [* tstr],            // canonical human name(s), e.g. "abc@def.com" (§3)
  prev:     ?bytes,             // hash of the previous Identity version (hash chain)
  ts:       u64,
  sig:      [+ bytes],         // one signature per suite in `suites`, over all of the above
}
```

Multi-suite support & PQ transition (normative). `suites` is a *set*, preference-ordered, so an identity can hold classical and PQ keys simultaneously during migration. Rules:

- A **sender MUST use the highest suite both parties support** (intersection of the sender's supported suites and the recipient's `Identity.suites`); if the intersection is empty, delivery fails closed (no silent downgrade).
- During transition the `Identity` MUST carry a signature under **every** suite in `suites` (`sig` is a list), so a verifier trusting either the classical or the PQ key can validate the object — this is what lets the network migrate without a flag day.
- Per-message suite is negotiated at **KeyPackage granularity** (§5.3): a KeyPackage advertises its suite, and the sender selects a recipient KeyPackage of the chosen suite. `Envelope.suite` records the suite actually used.
- A verifier MUST reject an `Identity` whose highest offered suite it cannot validate rather than fall back silently.

`prev` chains versions into a tamper-evident history mirrored in the transparency log (§3). A verifier accepts an `Identity` only if every `sig` validates under the corresponding key in `iks` and the chain is consistent with what it has previously pinned/seen.

1.4 Recovery policy (and rotating the recovery methods)

Recovery is **not** baked in at setup; it is a first-class, versioned, signed object so the recovery methods themselves can be rotated.

```
RecoveryPolicy {
  suite:     u8,
  ik:        bytes,
  version:   u64,
  methods: [+ RecoveryMethod],
  recover_threshold: Threshold,   // what regains access
  rotate_threshold:  Threshold,   // higher bar to change THIS policy
  prev:      ?bytes,             // hash chain
```

```
  ts:        u64,
  sig:       bytes,             // by IK, OR satisfied rotate_threshold quorum (reactive)
}

RecoveryMethod = PhraseMethod / DeviceMethod / SocialMethod
PhraseMethod { type:"phrase", recovery_key: bytes }            // key derived from a BIP39-style
DeviceMethod { type:"device", device_key: bytes, label: tstr }
SocialMethod { type:"social", guardians: [+ bytes], threshold: u8 }  // Shamir shares

Threshold = { any_of: [+ MethodPredicate] }   // e.g. 1 phrase OR 2 devices OR 2 guardians
```

Rules:

1. **Authenticated.** Every version is signed by `IK` (proactive) or by satisfying the current `rotate_threshold` (reactive, mid-recovery). Unauthenticated policy changes are invalid.
2. **`rotate_threshold` > `recover_threshold`.** A single compromised factor may (perhaps) recover access but MUST NOT be able to unilaterally rewrite the policy and lock the owner out.
3. **Real revocation.** Rotating a method out MUST re-key the underlying secret, not merely edit a list:
   - rotate **phrase** → re-wrap the identity secret under the new phrase-derived key; old phrase becomes useless;
   - change **guardians / threshold** → **redistribution / resharing** (a fresh access structure, Desmedt–Jajodia style), not merely proactive refresh; old shares fall below threshold and cannot reconstruct;
   - remove **device** → rotate any identity/recovery material it held.

   Recommended primitives (grounded):
   - Use **Verifiable Secret Sharing** (Feldman or Pedersen VSS), **not** plain Shamir, so guardians can detect a corrupt share or a cheating dealer at reconstruction (hostile-guardian threat model). Plain Shamir has no cheating detection.
   - **Proactive Secret Sharing** refreshes shares under the *same* (M,N); **redistribution** changes the access structure (add/remove guardians, change threshold). Use the correct one per operation.
   - Prefer **SLIP-0039** for the mnemonic Shamir encoding (purpose-built: two-level groups, checksums, passphrase) over hand-rolling BIP39 + Shamir.
   - **Strongly consider FROST (RFC 9591)** threshold Ed25519 signatures so guardians *authorize* recovery **without ever reassembling the secret key in one place** — eliminating the single-point-of-compromise moment that Shamir reconstruction creates. This is the preferred design when the recovery secret is a signing key.
4. **Logged & detectable.** Every version is published to the transparency log; the owner's other devices monitor it and MUST alert on a change they did not initiate (intrusion detection, §3.5).

Recovery flows

- **Proactive rotation** (owner has access): publish `version+1` signed by `IK`. Routine hygiene; do it whenever a factor is suspected compromised.
- **Reactive recovery** (owner lost access): satisfy `recover_threshold` to reconstruct `IK` (or a fresh `IK` authorized by the old under `rotate_threshold`), then immediately publish a new policy that invalidates the compromised factor.

The bottom turtle

There is a foundational anchor (root phrase + social quorum). It is rotatable but requires the strongest `rotate_threshold`. Losing `IK` **and** enough recovery factors simultaneously is unrecoverable — this is the one irreducible risk, mitigated only by good recovery UX (multiple independent, redundant, rotatable factors so no single loss is fatal).

1.5 Key rotation

To rotate `IK` (compromise, or scheduled PQ migration):

1. Generate `IK'`.
2. Publish an `Identity` version whose `sig` is by the **old** `IK` and whose body authorizes `IK'` (a cross-signed `KeyRotation` record: `old_ik` signs `new_ik` + `reason` + `ts`).
3. From then on `IK'` is authoritative; the transparency log's chain proves continuity.
4. Re-issue device certs and KeyPackages under `IK'`. Correspondents update the pin by following the signed chain (§3.4).

A verifier MUST accept `IK'` only via a valid chain from a previously-pinned `IK`, or via an explicit out-of-band re-verification.

1.6 Identity (name) migration — never losing your address

Because **key is identity**, losing a domain is a change of *name*, not of identity.

A **MoveRecord** rebinds the human name while preserving the key:

```
MoveRecord {
  suite:   u8,
  ik:      bytes,
  from:    tstr,          // "abc@old.com"
  to:      tstr,          // "abc@new.com"  (or a self-sovereign name, §3.6)
  ts:      u64,
  prev:    ?bytes,
  sig:     bytes,         // by IK
}
```

Distribution (all three):

1. **Mesh/DHT** — the key→location record and `Identity.names` carry the new canonical name.
2. **Transparency log** — an auditable, ordered record of the move (and of the key discontinuity at the old name, defeating a squatter who later registers `old.com`).
3. **Push to contacts** — a signed MOTE announcing the move.

Because existing contacts **route by key via the mesh** and verify the `MoveRecord` against `IK`, they follow the owner automatically and cannot be redirected by a forged move. Only *new* contacts who know only the abandoned name are affected — the same, unavoidable tradeoff as giving up any domain, but the *identity and all existing relationships survive*.

Optional maximal durability: to make the literal address itself un-loseable and un-seizable, bind the name in a self-sovereign name backend instead of (or in addition to) DNS (§3.6). This is the only place DMTAP admits a name-chain, and it is confined to the name layer.

1.7 Object summary

Object	Signed by	Chained	In transparency log
`Identity`	IK	yes (`prev`)	yes
`DeviceCert`	IK	via Identity	via Identity
`RecoveryPolicy`	IK or rotate-quorum	yes	yes
`KeyRotation`	old IK → new IK	via Identity chain	yes
`MoveRecord`	IK	yes	yes

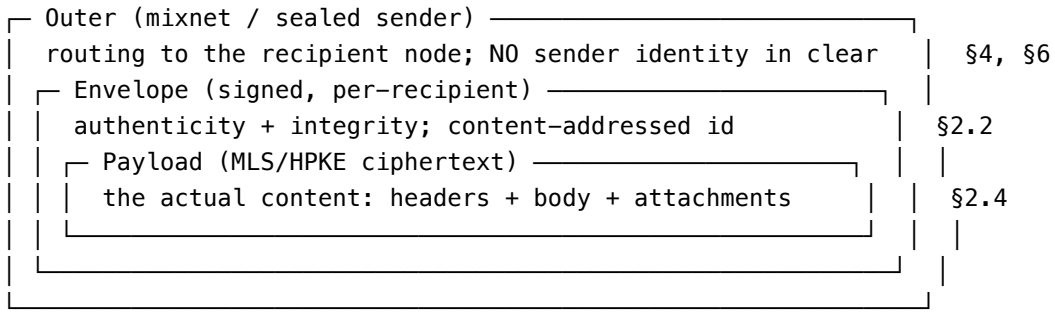
All identity-lifecycle operations share one machinery: **signed, versioned objects, threshold-gated changes, and the transparency log as the audit trail** (§3).

2. The MOTE Object

A **MOTE** (the atomic unit of DMTAP) is a signed, encrypted, content-addressed message object. Mail, chat messages, file-share announcements, group events, and identity announcements are all MOTEs — one format, rendered differently by clients (§5, §8).

2.1 Layered structure

A MOTE has three nested layers, each serving a distinct purpose:



The **outer** layer is what mix nodes and relays see: an onion-wrapped, constant-length (padded) packet with no clear-text sender (sealed sender, §6). The **envelope** provides authenticity to the recipient. The **payload** is the end-to-end-encrypted content.

2.2 Envelope (CBOR)

```
Envelope {
  v:         u8,             // format version (0)
  suite:     u8,             // algorithm suite (§1.1)
  id:        bytes,          // content address of `ciphertext` (§2.2, hash-agile)
  to:        DeliveryTag,    // routing target: recipient key, group id, or blinded tag (§2.2a)
  epoch:     ?bytes,         // MLS epoch / group context ref, if group (§5)
  ts:        u64,            // sender timestamp (ms epoch)
  kind:      u8,             // message kind (§2.3)
  keypkg:    ?KeyPackageRef, // present iff this initiates an MLS session (async join, §5.3)
  challenge: ?ChallengeResponse, // anti-abuse proof for cold senders (§2.2b, §9) —
                                 //   verifiable WITHOUT decrypting `ciphertext`
  ciphertext: bytes,         // MLS or HPKE sealed Payload (§2.4)
  sender_sig: bytes,         // detached sig by an EPHEMERAL per-message key over
```

```
                                //   (id, to, ts, kind, challenge); gates abuse, reveals no identity
}
```

- **`id`** — content address = a 1-byte **hash-algorithm prefix** (multihash-style, for agility) followed by the digest. v0 default is **BLAKE3-256** (128-bit collision resistance, equal to SHA-256; fast Merkle/XOF structure ideal for chunking). BLAKE3 is cryptographically sound but **not FIPS/IETF-standardized** (the IETF draft expired); the agility prefix lets an implementation migrate to SHA-256/SHA-3 where compliance requires it without changing the address format. Pin the exact BLAKE3 mode + 256-bit output. Content addressing gives deduplication, integrity, and cacheability for free; identical ciphertext shares an `id`.
- **Signature placement.** For sealed-sender messages the *authenticating* signature and the sender's identity live **inside** the encrypted payload (`Payload.from`, `Payload.sig`), so intermediaries never see who signed. `sender_sig` in the envelope, when present, is a detached signature by an *ephemeral* per-message key (unlinkable) used only for spam/abuse gating at the recipient (§9); it does not reveal identity.
- **`epoch`** ties a message to an MLS group epoch so the recipient selects the right key.

2.2a Delivery tag (`to`) and recipient blinding

`to` is a **`DeliveryTag`**, one of:

- the recipient's **identity key** (default, simplest); or
- a **group id** (for MLS group messages, §5); or
- a **blinded delivery tag** — a per-contact value `BT = HKDF(shared_secret, epoch_day)` derived from a secret established at first contact, which the recipient's node recognizes but which is **unlinkable across time and across observers** to the recipient's persistent key.

Blinded tags are RECOMMENDED for the `private` tier. **Honest limit (reconciled with §6.4):** even a blinded tag does not hide *that a packet was delivered to a particular node* from the final mix / an observer of the recipient's link — an always-on node has a stable network presence (§6.4). Blinding removes the *persistent-key* linkage in the envelope; it does not remove last-hop delivery observability, which §6.4 addresses with node/identity decoupling and recipient-side cover traffic. Implementations MUST NOT present blinded tags as full recipient anonymity.

2.2b Anti-abuse challenge (`challenge`)

`challenge` is an optional `ChallengeResponse` carrying the sender's proof for cold contact (§9): an **ARC token**, a **proof-of-work solution**, a **postage stamp**, or a **vouch**. It is placed in the *envelope* (not the payload) so the recipient can evaluate abuse policy **without decrypting** — see the validation order in §2.7. Known contacts omit it (their MOTEs are accepted on the fast path). See §9 for the grammar, issuer-trust rules, and each proof type.

2.3 Message kinds

```
0x00  mail           long-form message (email semantics)
0x01  chat           short message (chat semantics)
0x02  reaction       emoji/ack on a referenced MOTE
0x03  edit           supersede a referenced MOTE
0x04  redact         request deletion of a referenced MOTE
0x05  file_offer     manifest + key for a content-addressed file (§5.5)
0x06  group_event    MLS handshake / membership change (§5.3)
0x07  receipt        delivery/read receipt (opt-in; §6)
```

```
0x08  presence        ephemeral presence/typing (opt-in, off by default; §6)
0x09  identity        Identity/Move/RecoveryPolicy announcement (§1)
0x0a  system          protocol control (capability negotiation, §10)
0x40–0x7f  reserved for extensions (§10)
```

Kinds `mail` and `chat` differ only in default client rendering and default privacy tier (§6): `mail` defaults to the `private` tier, `chat` MAY use the `fast` tier when both parties are online. Both are the same object over the same transport.

2.4 Payload (CBOR, encrypted)

The plaintext that is sealed into `Envelope.ciphertext`:

```
Payload {
  from:     bytes,          // sender identity key (IK) — revealed only to the recipient
  sig:      bytes,          // IK (or device key) over the canonical payload hash
  headers:  Headers,
  body:     Body,
  refs:     [* bytes],      // ids of MOTEs this replies to / references (threading)
  attach:   [* Attachment], // §2.5
  expires:  ?u64,           // requested expiry (client-enforced deletion)
  fs_ratchet: ?bytes,       // forward-secrecy ratchet material (§5.2)
}

Headers {
  thread:   ?bytes,         // stable thread/conversation id
  subject:  ?tstr,          // mail only
  mime:     ?tstr,          // content type of `body`
  cc:       [* bytes],      // additional recipient keys (fan-out is per-recipient MOTEs)
  ext:      { * tstr => any },   // extension headers (§10)
}

Body = tstr / bytes         // text or opaque MIME
```

Only the recipient (or group members) can decrypt `Payload`, so **all sender identity, subject, recipients, threading, and content are hidden from the network** — this is what sealed sender + payload encryption buys.

2.5 Attachments and files

Small attachments MAY be inlined into `Payload.attach`. Larger files MUST be referenced by a content-addressed **manifest** and transferred out-of-band (direct/fast tier, §4.5), which is what makes DMTAP a file-share of arbitrary size (no protocol cap).

```
Attachment {
  name:     tstr,
  mime:     tstr,
  size:     u64,
  inline:   ?bytes,         // present iff small
  manifest: ?ManifestRef,   // present iff large (§5.5)
  key:      bytes,          // per-file content key (recipient decrypts chunks)
}

ManifestRef { id: bytes, size: u64, chunks: u32 }   // BLAKE3 Merkle-DAG root (§5.5)
```

The manifest lists chunk hashes (a BLAKE3 Merkle DAG). Chunks are fixed-size, individually encrypted and content-addressed, enabling **resumable, parallel, swarmed, deduplicated** transfer. Only the manifest + `key` travel in the (private) MOTE; the bulk chunks travel direct (§4.5). See §5.5 for the full file model.

Size tiers (normative threshold, reconciles §2.5 / §4.5 / §6.5). A file is handled by one of three paths, by size:

Tier	Size	Path
inline	1 padded packet (v0: 64 KiB after padding)	in `Attachment.inline`, inside the MOTE
normal	> inline, 4 chunks (v0: 4 MiB)	manifest in MOTE; **chunks also routed via th**
large	> normal	manifest in MOTE; **chunks via the fast/onion**

The v0 numeric thresholds (64 KiB / 4 MiB) are parameters (§ numeric appendix) and MAY be tuned; the three-tier model is normative. This removes the earlier binary small/large ambiguity.

2.6 Delivery semantics

```
deliver(outer_mote)   → recipient node receives, unwraps outer, verifies envelope,
                        decrypts payload, stores, and returns:
ack(id)               → recipient confirms receipt of MOTE `id`.
```

- **Durability = the sender's node retries** until `ack`, with exponential backoff and an `expires`-bounded deadline. The mixnet/relay holds nothing durably.
- **Deduplication.** A recipient that already holds `id` acks immediately without re-processing.
- **Ordering.** MOTEs are not globally ordered; `ts` + `refs` + (for groups) MLS `epoch` provide causal/threading order. Clients MUST tolerate out-of-order and duplicate delivery.

2.7 Validation (recipient MUST)

Validation is ordered **cheapest-and-anonymous first**, so a flood of cold junk is rejected *before* any expensive asymmetric decryption (a decryption-DoS defense). On receipt a node MUST, in order:

1. Reject unknown `v` / `suite` (fail closed).
2. Verify `id` matches the content address of `ciphertext` (§2.2); drop on mismatch.
3. Verify `sender_sig` over `(id‖ to‖ ts‖ kind‖ challenge)` under its ephemeral key (cheap; no decryption). Drop on failure.
4. **Resolve `to`** to this node (or a group it belongs to). If `to` does not resolve, drop.
5. **Classify the sender** by `to`/pinning state: a **known contact** (fast path) vs an **unknown/cold sender**.
6. **For cold senders, enforce anti-abuse policy (§9) NOW, before decryption**, using the `challenge` field (ARC token / PoW / stamp / vouch) — all checkable without decrypting. Reject or defer per §9.2 / §2.7a if the challenge is absent or insufficient.
7. Decrypt `ciphertext` (MLS epoch key or HPKE to recipient key); drop on failure.
8. Verify `Payload.sig` under `Payload.from`; verify `from` matches the pinned identity for a known contact, or TOFU-pin on first contact (§3.4). For a cold sender whose `from` is now revealed, re-apply block/allow lists.
9. Apply `expires`, `refs`, `kind` semantics; store; `ack`.

Known contacts MAY skip step 6 (they are pre-authorized). Only known-contact MOTEs reach decryption on the fast path; unknown senders must pass the anonymous abuse gate first.

2.7a Outcome of a failed/absent challenge (normative)

To reconcile §2.7 and §9.2, the disposition is by *degree*:

- **Invalid or forged** `challenge`, or failed `sender_sig` / `id` → **discard silently**, no user-visible effect, do not `ack` (except a duplicate `id`, which is acked).
- **Absent or below policy threshold** (a cold sender with no/weak proof) → **defer to a "requests" area** (not the inbox), rate-limited, never silently dropped and never surfaced as a normal message. The user MAY promote the sender (which pins them as a contact).

Implementations MUST NOT deliver an unproven cold MOTE to the inbox, and MUST NOT silently discard a well-formed-but-under-threshold one without the requests-area affordance.

2.8 Why this shape

- **Content-addressed** → dedup, integrity, caching, and file chunking all fall out of `id`.
- **Three layers** → clean separation of *routing privacy* (outer), *authenticity* (envelope), and *confidentiality* (payload).
- **Sealed sender** → the network sees ciphertext to an opaque destination and nothing else.
- **One object, many kinds** → mail, chat, files, groups, and identity share the format, so the transport, store, and crypto are built once (§5).

3. Naming & Directory (name → key)

Your **key is your identity and the only root of trust**; a `name@domain` is the everyday, human-facing pointer to it; the mesh finds you by key. This section covers the **stable** binding: how `abc@def.com` resolves to an identity key, how that binding is made tamper-evident, and how it degrades safely.

The layering (read this first). Three layers, never conflated:

- **The key is identity and proof.** Authenticity is *always* the key (`IK`, §1.2) — nothing else. A name is only a pointer to it.
- **DNS (and any name backend) is *discovery*, never proof.** It tells you which key *claims* a name; it does not attest it. A compromised registrar can lie about the pointer, so the pointer is never trusted on its own.
- **Key transparency (KT) makes the `name → key` binding tamper-evident**, so a silent key swap by DNS/registrar/directory is *detectable* (§3.5).
- **Pinning (TOFU) makes discovery a one-time event.** After first contact you route by the pinned key via the mesh (§4); DNS is not consulted again unless the signed identity chain says to. A later DNS/registrar compromise cannot redirect an existing relationship.

Stable anchor, rotatable keys (the real future-proofing). The human name binds to a **stable identity anchor — the root identity key `IK` (§1.2)** — *not* to a rotatable operational key. Day-to-day signing/device keys rotate *under* the identity without changing the name; even `IK` itself can rotate, and the suite can migrate to post-quantum, without the name changing — bridged by the signed hash chain (§1.5) and, for a change of the anchor's name, by aliases + a signed `MoveRecord` (§1.6). So a `name@domain` **survives ordinary key rotation and PQ migration**: the `name → key` indirection is exactly what lets the key underneath change while the address people type stays the same. `IK` rotation is the rare migration event, not the common case.

3.1 Roles (who holds what)

- **The key (`IK`)** is the identity and the sole trust root. Everything below only *points* to it; nothing below can prove authenticity on its own.
- **DNS** (we do not run it — registrars/operators do) holds the stable `name → key` *pointer*. It is discovery: static and cacheable. It MUST NOT hold location (that is the mesh, §4), and it is **never** the proof — KT + pinning are.
- **Key transparency (KT)** makes the pointer *auditable* — since DNS is a trusted third party the owner does not control, KT lets a silent key swap be *detected* (§3.5).
- **The mesh DHT** holds the dynamic `key → location` record (§4).

3.2 DNS records

For `abc@def.com`, the resolver queries:

```
abc._dmtap.def.com.  IN  TXT  "v=dmtap1; suite=1; ik=<base64url IK>; id=<hash of Identity §1.3>;
                                 kt=<KT log URL>; keypkgs=<KeyPackage bundle locator §5.3>"
_dmtap.def.com.      IN  SVCB 1 . ( ... )     ; optional service params, KT anchors
def.com.             IN  MX   ...             ; only if a legacy gateway serves the domain (§7)
```

- `ik` is the identity public key (or a hash of `Identity` that the mesh resolves to the full object). `id` pins the current `Identity` version (§1.3).
- Multiple `ik`/suite entries MAY appear during PQ migration (§1.1).
- DNSSEC SHOULD be enabled; it is not sufficient alone (hence KT).

3.3 Resolution

```
resolve(name):
  1. DNS TXT/SVCB lookup for name → { iks, id, kt, keypkgs }
  2. (first contact) verify against KT (§3.5): fetch a signed tree head + inclusion proof for
     this identity, and confirm no newer version supersedes it (rollback defense).
  3. fetch full Identity (§1.3) from the mesh by `id`; verify sig chain
  4. PIN (name → iks, id) locally (TOFU); offer out-of-band verification to upgrade the pin
  5. thereafter: route by key via the mesh (§4); DNS is not consulted again unless the
     pinned Identity chain says to (rotation/migration)
```

Fail-closed on unreachable KT (normative, §12 finding). If KT is unreachable, partitioned, or censored at **first contact**, the client MUST NOT silently TOFU-pin an unverified key — it MUST either refuse to pin (block) or hard-warn and require explicit user acceptance, and MUST prefer out-of-band verification. Silent downgrade to unverified TOFU is prohibited, because it enables an attacker to replay an old-but-validly-signed `Identity` (e.g. from before a legitimate rotation) precisely under the network conditions that make KT unreachable. Once a key is pinned, later KT unavailability does not block routing (the relationship is already key-based).

DNS is the **front door, used once**. After first contact the relationship is key-based and DNS-independent — a later DNS/registrar compromise cannot silently redirect an existing contact.

3.4 Trust on first use (TOFU) + pinning

v0 trust model:

- On first resolution, **pin** `(name → ik, id)`.

- Follow the signed `Identity` hash chain (§1.3–1.6) for rotations and migrations; accept a new key only via a valid chain from the pinned key.
- Offer **out-of-band verification** (safety-number / QR comparison of `ik`, §3.4.1) to upgrade a TOFU pin to a verified pin.
- A key that changes *without* a valid chain MUST raise a security warning, never silently update.

Honest limit: a MITM at the *very first* contact (before KT is consulted or before OOB verification) can substitute a key. KT (§3.5) closes this; OOB verification closes it immediately for high-value contacts.

3.4.1 Safety numbers (out-of-band key verification)

Out-of-band verification compares the **key**, not the name — it is the strongest trust upgrade and the one thing that closes a first-contact MITM immediately.

- A **safety number** (Signal-style) is a deterministic function of the parties' `IK`s (the fingerprint of the identity keys). Two contacts confirm they see the same value out-of-band — in person, over a trusted channel, or by scanning a **QR code**.
- **This is verification, not an address.** A safety number / fingerprint is *never* used to route or reach someone; it only confirms that the key you pinned is the key you meant. It does not appear in `Identity.names` and cannot be typed at to send mail.
- **Word rendering (optional).** Because digit strings are error-prone to compare aloud, a fingerprint MAY be rendered as a **word sequence** for easier human comparison: `words(fingerprint, wordlist)` over a curated ~**1024-word, language-agnostic** list (short, pronounceable across major languages, no homophones/confusables/offensive collisions), 10 bits/word, with a folded **checksum** so a misheard word fails closed rather than comparing as a different key. **Proquints** (pronounceable 5-char syllables, 16 bits each) are an allowed language-neutral alternative encoding of the same bits.
- The word encoding exists **only** for this verification role — a comparison aid for confirming a key. It is deliberately **not** an address: DMTAP does not name people by their key digits. Because the safety number is taken over the full identity key, it carries the key's full strength; there is no separate truncated-name address to grind.

3.5 Key transparency (KT)

Status: designed-in, v0 ships a minimal form; full CONIKS/Key-Transparency-style logs are a v1 hardening.

- The owner's identity events (`Identity`, `RecoveryPolicy`, `KeyRotation`, `MoveRecord`, §1) are appended to an **append-only Merkle log**.
- Verifiers obtain **signed tree heads** and **inclusion/consistency proofs**; they gossip tree heads to detect **split-view/equivocation** (the log showing different histories to different observers).
- The owner's own devices **monitor** the log for the owner's entries and MUST alert on any change the owner did not initiate — turning KT into **identity intrusion detection**.
- Logs MAY be federated; a verifier trusts a *set* of logs and requires consistency across them. No single log is authoritative.

v0-minimal: a single append-only log with signed heads and inclusion proofs; monitoring and gossip-based equivocation detection land in v1.

3.6 Optional: self-sovereign naming (un-loseable addresses)

DNS names can expire or be seized. For a name that is **un-loseable and un-seizable** as long as the owner holds `IK`, DMTAP defines a pluggable **name backend** interface. A conforming backend maps `name → ik` such that only `IK` can update the binding and it cannot lapse without the owner's action. A name-chain (ENS-style) is one such backend.

This is the **only** place DMTAP admits a blockchain, it is **optional**, and it is confined to the name layer — nothing else in DMTAP depends on it. Resolution (§3.3) treats a self-sovereign name identically to a DNS name once `ik` is obtained.

3.7 Private lookups

To keep discovery metadata-private, name→key lookups SHOULD be routed **through the mixnet** (§6), so neither the DNS resolver nor a KT log learns *who* is asking. This replaces heavier private-contact-discovery schemes for v0; stronger schemes are a v1 option.

3.8 Onboarding tiers (day-one setup)

The identity is a **key**; DNS is a generated, auto-managed *projection* of it, never hand-edited. There are three onboarding tiers. A conformant client SHOULD default to Tier B (a provider-issued `name@domain`) and never require a user to author DNS.

Tier A — no domain (pure DMTAP)

Identity = key; the human name is a provider directory name or a self-sovereign name backend (§3.6) that resolves directly to the key. **No DNS, no domain.** DMTAP DMTAP only (the legacy world cannot resolve the name). Setup: generate key → claim a directory name → join mesh. Most users instead pick Tier B so their address also works for legacy email.

Tier B — gateway-owned domain (default; zero user DNS)

The user claims `alice@gw.example`; the **gateway operator owns `gw.example` and maintains all legacy records once, for all users**:

```
gw.example         MX      → gateway
gw.example         TXT     SPF (gateway sending IPs)
sel._domainkey     TXT     gateway DKIM key
_dmarc.gw.example  TXT     DMARC policy
+ *@gw.example name→key directory (service or programmatic/wildcard records)
```

The **user touches zero DNS.** DMTAP-native delivery is key-based; legacy interop works immediately because the shared domain is pre-configured and its IPs are warmed. Per-user legacy setup = none. This is Gmail-grade onboarding: you get an address, not a domain — a provider-issued `you@gw.example` that is a full, first-class `name@domain` (§3.9.1).

Tier C — vanity custom domain (auto-configured)

For `alice@yourbrand.com`, the domain's own DNS MUST carry MX/SPF/DKIM/DMARC (DMARC alignment is defined on the `From:` domain — unavoidable while legacy exists). But it MUST NOT be hand-edited: the provider auto-publishes the full record set via a **registrar API / the Domain Connect standard** or by hosting the domain's DNS. The user approves once; records self-update on key rotation.

DKIM delegation / DMARC alignment (all legacy-facing tiers)

The domain publishes the **gateway's DKIM public key** at `sel._domainkey.<domain>`; the gateway signs outbound with `d=<domain>`, so DKIM passes and **DMARC aligns** — Gmail/Outlook accept it as ordinary authenticated mail. The gateway holds only a delegated *DKIM* key, never the user's DMTAP identity key (§7.3). Inbound legacy uses the domain's MX → gateway (§7.2).

What cannot be skipped

Exactly one thing is irreducible: **some** operator must configure **one** legacy domain properly and **warm its sending IPs** (weeks). This is a one-time, per-domain operator task, amortized across all users on that domain — never a per-user task.

3.9 Names (one identity, many pointers)

The identity is always the **key**; every name is a memorable pointer to it, and one identity MAY hold several names at once — all resolving to the same key. DMTAP has **one recommended, headline form** and a few optional extras. This is deliberately *not* a ladder of equals.

Zooko's triangle, stated plainly. A name cannot be simultaneously *human-meaningful*, *global*, and *authority-free*. DMTAP's choice: names are human-meaningful and — through DNS federation — global, and it pays for that with an authority (DNS/registrar). But it **neutralizes** that authority by keeping the key the sole trust root, KT-auditing the binding (§3.5), and pinning on first use (§3.4): the authority can help you *discover* a key, it can never *forge* one undetected. The alternative corner (a flat authority-free name derived from the key) is not offered as an address — it is unwieldy, and it would name people by a rotatable artifact rather than the stable identity.

3.9.1 `name@domain` — the primary human address

`name@domain` is the everyday, recommended, headline address. It comes in two flavours, one format:

- **Provider-issued** — `you@envoir.org` and the like (Bluesky-style): the majority who don't own a domain get a familiar, handle-shaped address for free from a provider, with zero DNS work (Tier B, §3.8).
- **Your own domain** — `you@yourbrand.com`, for full sovereignty (Tier C, §3.8).

Why this is the headline:

- **It survives key rotation and PQ migration.** The name binds to the stable anchor `IK`, not to a rotatable key; the key underneath can rotate or migrate to a PQ suite and the address is unchanged (`name → key` indirection, §1.5–1.6). This is the real future-proofing.
- **It's federated — no global-namespace squatting.** Uniqueness is *per-domain*, so there is no single global registry to squat or race: `alice@a.org` and `alice@b.org` are different people, and neither had to win a landrush.
- **One familiar format**, already understood by every human and every mail system.
- **It interoperates with legacy email for free** — the shared or owned domain carries MX/DKIM/DMARC (§3.8, §7), so `name@domain` is reachable from Gmail/Outlook on day one.

Authenticity is still the key: a `name@domain` is resolved (§3.3), KT-audited (§3.5), and pinned (§3.4) — the domain only points, it never proves. For a name that cannot expire or be seized, the same pointer can live in a self-sovereign name backend instead of DNS (§3.6), with resolution unchanged.

3.9.2 `@handle` — optional global registry (opt-in)

A flat, global `@handle` namespace is **optional, not the headline.** For handle-like UX, DMTAP prefers the **provider-federated** `you@provider` form above, which gives the same short-name feel *without* a single global namespace to fight over.

A conforming client MAY additionally offer an opt-in `@handle` directory. Because a *chosen, globally-unique* name requires arbitration (Zooko), the directory is a thin authority that:

- assigns each handle once (first-come-first-served) with an **anti-squat cost** (a small fee or proof-of-work, since assignment gives uniqueness but not scarcity);
- publishes `handle → key` to a **key-transparency log** (§3.5), so it is *auditable, not trusted* — it cannot silently repoint a handle without detection;
- **cannot hijack existing relationships** — contacts route by the pinned key (§1.6), so the handle is only an introduction.

Handles are normalized (NFC, lowercase, collapse consecutive dots; dots allowed as cosmetic separators, e.g. `@this.is.my.name`) and confusables/homoglyphs are reserved. The directory SHOULD be a **federated consortium running BFT consensus** (no single owner of the namespace, no chain) rather than a single operator; a name-chain (§3.6) is a further option.

Honest limit (why it is not the default): a single global namespace **reintroduces exactly the global squatting and impersonation that domain-federation avoids** — every desirable handle becomes a landrush and a phishing target (`@paypa1` vs `@paypal`). That is the unavoidable price of a flat global name, and it is why `name@domain` is the recommended form and `@handle` is an explicit opt-in.

3.9.3 Petnames (local)

A user assigns **petnames** locally to contacts ("Mom" → a key). Petnames are human + zero-authority but *local-scope only* (not global), and never leave the device cluster (they are a social-graph artifact and MUST be stored encrypted at rest with the mailbox).

3.9.4 Aliases & subaddressing (one identity, many addresses)

For feature parity with legacy email, **one identity MAY hold multiple names** — all resolving to the same key — and support subaddressing:

- **Aliases.** An identity's `Identity.names` (§1.3) is a *list*: a user can hold several `name@domain` addresses (provider-issued and/or own-domain) at once, optionally an `@handle`, all pointing to the same key. Mail to any of them arrives in the same mailbox.
- **Retaining a legacy address.** Crucially, a **legacy email address you already own can be an alias**: point that domain's MX/DKIM at a gateway (§7) and add the address to `Identity.names`, so people who only know your old `you@oldprovider.com` still reach you (via the gateway → MOTE), while you migrate. This is the practical migration path — you don't lose your old address; it becomes one of your DMTAP aliases.
- **Subaddressing (plus-addressing).** `you+tag@domain` (and, for handles, `@you+tag`) is supported: the `+tag` is preserved for client-side filtering/labeling but resolves to the same key. Catch-all (`*@yourdomain`) is a domain-owner option (tier C).
- **Security:** every alias resolves to the *same key*, so an alias cannot be used to impersonate — authenticity is always the key, not the name. Aliases are published/audited via the same KT (§3.5); adding/removing an alias is a signed `Identity` version. A legacy-alias's inbound mail is marked *legacy-origin* (not E2E before the gateway, §7.2), so the user sees which messages came the old way.

4. Transport: Mesh, Mixnet, Delivery

The mesh finds you and moves ciphertext to you; the mixnet hides who is talking to whom. The node **is** the mesh — relay and mix roles are capabilities of the node binary, not separate services.

4.1 Substrate: libp2p

DMTAP builds on **libp2p** rather than reinventing P2P:

- **Kademlia DHT** — peer routing and the `key → location` record store (see §4.2 for its security limits — it is the weakest link).
- **Circuit relay v2** — reachability for NAT'd nodes (the "relay" role).
- **AutoNAT v2 + DCUtR (hole punching)** — direct connections where possible.
- **Noise** (or TLS 1.3) — hop-by-hop channel security (in addition to end-to-end MOTE encryption; QUIC carries TLS 1.3 natively).
- **QUIC / TCP / WebSocket / WebRTC / WebTransport** transports.

A node dials **outbound** and holds connections, so CGNAT/dynamic-IP nodes are reachable.

Roaming — honest note: identity is a persistent keypair, not an IP. When a node's address changes, roaming is carried primarily by **re-publishing the location record (§4.2) and peers re-dialing**, not by QUIC connection migration. QUIC migration (RFC 9000) *can* preserve some live connections, but the Rust QUIC stack (`quinn`) has no multipath and address-change handling is imperfect; do not rely on it for seamless roaming.

4.2 The `key → location` record (DHT)

```
LocationRecord {
  ik:        bytes,        // identity key (DHT key = hash(ik))
  peer_id:   bytes,        // libp2p peer id (may be per-epoch / unlinkable, §6)
  addrs:     [* multiaddr],// current reachability hints (may be relay circuits, mix addrs)
  ttl:       u64,
  ts:        u64,
  sig:       bytes,        // signed by a device key
}
```

The record follows the **IPNS pattern**: a self-certifying **value record** (DHT key = hash(ik)), signed, with a **monotonic sequence number + EOL/TTL** to defeat rollback/replay, stored at the **K closest peers**, and **aggressively republished** (DHT record lifetimes are short — hours — so staleness is a real failure mode). Use value records, not provider records.

CAUTION — signing does NOT stop eclipse attacks (the DHT is the weakest link)

Signing authenticates record *content*, not *routing*. Because a libp2p PeerId is `hash(pubkey)`, an attacker can cheaply generate IDs closest (XOR-distance) to a target key, fill honest routing tables, and control all lookups for that key — returning nothing or an *old, still-valid signed* record (censorship / rollback) **without forging anything**. This is a Sybil/eclipse attack at the routing layer, and it is the single most attackable part of DMTAP. Mitigations DMTAP REQUIRES/RECOMMENDS:

- **S/Kademlia disjoint-path lookups** (parallel, node-disjoint) and **IP-diversity caps** per k-bucket.

- **Aggressive republish** + accept only records with a newer sequence number (rollback defense).
- **The DHT is one discovery mechanism, not the root of trust.** Root of trust = the user's long-term keypair. Resolution order: (1) cached direct addresses, (2) **relay-reservation / rendezvous ("home relay") addresses**, (3) DHT as fallback. A fresh contact SHOULD have a non-DHT path (rendezvous/introduction) so it is not 100% dependent on a hostile public-DHT lookup.
- Closed/organizational deployments SHOULD use a **private DHT** (own protocol prefix) to shrink the Sybil surface.

4.3 Reachability ladder

Try in order; fall down only as needed:

1. **Direct** — IPv6, or IPv4 with port-forward/UPnP. No relay.
2. **Hole-punch** — DCUtR between two nodes (both online; always true for always-on boxes).
3. **Circuit relay** — a public-IP node relays; ciphertext-only, content-blind.

As IPv6 spreads, rungs 1–2 dominate and the relay role fades. Downtime is covered by sender-retry (§2.6) and optional **peer buffering** (buddy nodes hold ciphertext during an outage) — no central buffer required.

4.4 The mixnet (metadata privacy)

For the `private` tier, MOTEs are sent as **Sphinx-format**, constant-length, onion-wrapped packets through a sequence of **mix nodes** (Loopix/Nym-style):

- Each mix peels one layer; **no mix sees both sender and recipient**.
- Mixes add **randomized (Poisson) delays** and reorder, defeating timing correlation.
- Nodes emit **cover traffic** (loop + drop messages) at a steady rate so an observer cannot tell when real messages are sent/received. Cover rate is a **tunable knob** (§6).
- **Sealed sender** (§6.2): the sender identity is inside the payload, never in the outer packet.
- **Size padding**: MOTEs are padded to fixed buckets so size does not leak content.

Mix nodes are the same permissionless, content-blind contributor model as relays; incentive and Sybil-resistance are covered in §6.4 and §9.

Email's asynchrony is what makes full-strength mixing affordable: minutes of latency are acceptable for the `private` tier.

4.5 Bulk / file transfer

Large blobs (§5.5) MUST NOT traverse the mixnet (impractical bandwidth/latency). Instead:

- The **control MOTE** (`file_offer` : manifest + key) travels the **private** tier.
- The **chunks** transfer **direct** (fast tier), content-encrypted, **swarmed** from any holder (BitTorrent-style), resumable per chunk.

Honest tradeoff: the *fact and size* of a bulk transfer between two nodes is observable (direct), though the content is encrypted and the control message is private. See §6.

4.6 Privacy tiers

Tier	Path	Latency	Metadata privacy	Default for
`private`	mixnet + cover traffic	minutes	full (global passive)	mail, all control messages
`fast`	direct / low-hop mesh	sub-second	content only; graph observable	live chat (both online), bulk

Default is `private`. `fast` is opt-in per conversation/message. Choosing `fast` is itself metadata; clients SHOULD keep `private` as the standing default and cover it with cover traffic (§6).

4.7 Delivery state machine (sender)

```
QUEUED → SEALED (sealed sender + onion, §6) → IN_FLIGHT (mixnet/direct)
       → ACKED (done)  |  RETRY (backoff)  |  EXPIRED (drop, notify user)
```

Durability lives entirely in this sender-side queue. The middle is stateless.

4.8 Local, isolated, and delay-tolerant networks

DMTAP works in **remote environments with their own networks** — often *more easily than email*, because email needs an SMTP/IMAP server + MX + DNS, whereas two DMTAP nodes on the same network need no infrastructure at all.

- **Local discovery (mDNS).** Nodes on the same LAN discover each other via libp2p **mDNS** (`_p2p._udp.local`) with **zero configuration, no internet, and no central server**. A ship, a remote site, an air-gapped office, or a home LAN can run a fully functional local DMTAP mesh with nothing but the nodes. Private networks use a fingerprinted service name so they do not cross-discover.
- **Scope of the "easier than email" claim.** True specifically for the **local / same-network / isolated** case. Across the open internet, DMTAP still depends on the DHT/relays (§4.2–4.3), which have their own fragility.
- **Delay-tolerant store-and-forward is a DMTAP layer, not a libp2p feature.** libp2p is connection-oriented and provides no bundle/epidemic DTN routing. DMTAP REQUIRES its own store-and-forward: the sender-side retry queue (§4.7), **peer buffering**, and **sync-on-reconnect** of the device-cluster CRDT (§5.6). This is what lets an intermittently connected site queue locally and reconcile with the wider mesh when connectivity returns.
- **Radio transports (Bluetooth / Wi-Fi Direct / LoRa) are out of scope for v0.** libp2p ships no such transport (Briar's offline radio transports are its own Bramble code, not libp2p). Supporting them would mean writing a custom libp2p `Transport` — flagged as future work, not a v0 claim.

5. Messaging & Files (the unified substrate)

Mail, chat, and files are modes over one substrate: **MLS** (RFC 9420) for all session and group security, MLS **KeyPackages** for async initiation, and **content-addressed chunked blobs** for files. The same group primitive serves 1:1, group chat, email mailing-lists, multi-device, and shared file folders.

5.1 Why MLS everywhere

DMTAP standardizes on **MLS as the unifying crypto primitive**:

- **1:1** = a 2-member MLS group.
- **Group chat / mailing-list** = an MLS group.

- **Multi-device** = each of the owner's devices is a member (MLS handles this cleanly where pairwise ratchets get messy).
- **Shared file folder** = an MLS group over a set of manifests (§5.5).

MLS is **transport- and identity-agnostic** by design (RFC 9420, 2023; architecture in RFC 9750). Its abstract roles map onto DMTAP:

- **Delivery Service (DS)** → the DMTAP mesh/mixnet (§4).
- **Authentication Service (AS)** → DMTAP identity + KT (§1, §3). KT realizes the AS while *reducing* the trust placed in it — a good fit for a decentralized design.

MLS provides group forward secrecy and post-compromise security via the TreeKEM ratchet tree, with O(log n) member changes. v0 uses ciphersuites `0x0001` (`MLS_128_DHKEMX25519_AES128GCM_SHA256_Ed25519`) and `0x0003` (`…CHACHA20POLY1305…`); PQ migration uses MLS's own PQ ciphersuites (`draft-ietf-mls-pq-ciphersuites`, ML-KEM) and the MLS combiner (`draft-ietf-mls-combiner`), **not** a bolted-on external handshake.

CAUTION — epoch ordering is the hard part on a mesh

MLS trusts the DS for exactly one thing: **a total order on epochs** — Commits must be applied in an agreed order per group. A leaderless mesh has no natural serialization point, so **this ordering/consensus of Commits is the real difficulty of "MLS over P2P," not the crypto.** DMTAP REQUIRES that MLS **handshake** messages (Proposal / Commit / Welcome) travel over an **ordered, reliable channel** per group, while ordinary **application** messages (mail/chat/files) MAY travel over the reordering mixnet. Sending handshakes over the mixnet would stall or fork group state.

v0 ordered-channel: the group committer (normative). Each group has a **committer** — the node that serializes handshake messages into an append-only, hash-chained per-group log:

- **Selection:** the group creator is the initial committer. Committer identity is a signed field of the group state; every member knows it.
- **Rotation:** any member MAY be promoted to committer by a Commit (so committer role is not tied to one device's uptime). Committer SHOULD rotate on the current committer going offline past a timeout, or on member vote, via a Commit that all members apply.
- **Failover / liveness:** if the committer is unreachable, members hold pending Proposals and either wait for it to return or elect a new committer (a Commit referencing the last agreed log head). The hash-chained log makes a **fork detectable**: two Commits at the same log position with the same predecessor is proof of committer misbehavior, and members MUST halt and alert (analogous to KT equivocation, §3.5).
- **Censorship:** a committer can *stall* (withhold ordering) but cannot *forge* — every handshake is member-signed. A stalling committer is rotated out.

Metadata exposure (honest, §6.6 item 7). The committer necessarily sees **all handshake traffic for its group** — an explicit exception to the "no single node sees both ends" framing. This is bounded by: the committer sees only *membership-change* metadata (not application message content or the social graph outside the group), rotating the role spreads exposure, and application traffic still uses the mixnet. Groups needing stronger metadata protection SHOULD rotate committer frequently.

Track `draft-kohbrok-mls-dmls` (Decentralized MLS) for a fully leaderless replacement; until it lands, the committer model above is v0.

5.2 Forward secrecy & (non-)deniability

- **Forward secrecy** comes from MLS epoch advancement. Healing (post-compromise security) is **per-epoch/per-Commit**, which is coarser than the Signal Double Ratchet's per-message healing — using MLS for 1:1 (§5.1) is a deliberate *simplicity-vs-optimality* trade, not a strict improvement over Double Ratchet.
- **Deniability — honest correction.** MLS is **signature-based and therefore non-repudiable**: every LeafNode and every FramedContent (handshake *and* application messages) is signed with the member's signature key. RFC 9750 states plainly that *"MLS does not make any claims with regard to deniability."* DMTAP therefore **does NOT claim message deniability** at the MLS layer, and MUST NOT substitute shared-secret MACs for MLS's mandatory signatures (that would break RFC 9420 conformance). Deniability is an **explicit open design item**: if required, it must be engineered at another layer (e.g. exchanging the signature keys themselves over a deniable channel, per RFC 9750's note), or scoped to a pairwise/OTR-style sub-channel. Do not represent DMTAP messages as deniable in v0.

5.3 Async session initiation (MLS-native)

To message an identity whose devices are all offline, DMTAP uses **MLS's own async-join mechanisms** rather than bolting on a separate PQXDH/X3DH handshake (which would add a second protocol and a second security proof for no benefit when everything is already MLS):

1. Each device pre-publishes signed **KeyPackages** (located via `Identity.prekeys`, §1.3) to the mesh. A KeyPackage is MLS's prekey: identity/signature key, HPKE init key, and (for PQ) an ML-KEM init key via the MLS PQ ciphersuite.
2. The initiator **Adds** the offline member (a Commit) and sends a **Welcome** so the new member bootstraps group state on return; or the joiner uses an **External Commit** against the group's `GroupInfo` to self-join.
3. Post-quantum protection comes from the **MLS PQ ciphersuite / combiner** (§5.1), keeping a single key schedule.

KeyPackages are replenished by the owner's node; last-resort KeyPackages avoid exhaustion. (PQXDH-for-init would only be relevant if the 1:1 layer were Signal Double Ratchet rather than MLS; DMTAP chooses MLS everywhere, so it is not used.)

5.4 Message kinds in context

- `mail` / `chat` — the same MOTE; differ by default tier (§4.6) and client rendering.
- `reaction` / `edit` / `redact` — reference a prior MOTE by `id` via `refs`.
- `group_event` — MLS handshake messages (add/remove/update/commit).
- `receipt` / `presence` — opt-in, ephemeral, off by default (metadata-sensitive, §6).

5.5 Files: content-addressed, chunked, any size

A file is a **BLAKE3 Merkle-DAG manifest** over fixed-size encrypted chunks:

```
Manifest {
  id:        bytes,        // BLAKE3 root over chunk hashes (the content address)
  size:      u64,
  chunk_sz:  u32,          // e.g. 1 MiB
  chunks:    [+ bytes],    // ordered chunk hashes
  key:       bytes,        // file content key (delivered inside the MOTE, §2.5)
```

```
  suite:     u8,
}
```

Properties:

- **Any size** — no protocol cap; the file is bounded only by the owner's storage.
- **Resumable / parallel / swarmed** — fetch chunks from any holder, restart per chunk, BitTorrent-style; a popular file becomes *more* available as more nodes hold it.
- **Deduplicated** — identical chunks (and identical files) stored once by content address.
- **Streamable** — consume in manifest order before full download.
- **Integrity** — every chunk self-verifies against its hash; the manifest self-verifies against `id`.

Availability tiers (files reintroduce storage economics):

Tier	Mechanism	Cost
Best-effort	origin node online + swarm from holders	$0
Durable	encrypted peer-cache / replica	peer reciprocity
Always-available	paid replica / managed host	real storage cost

The always-on box gives best-effort/durable for free; "Dropbox-like always available" for large files is where a paid replica re-enters. Transfer uses the fast/direct tier (§4.5).

5.6 Multi-device (the personal cluster)

An owner's devices form an MLS group (`caps` per DeviceCert, §1.2). The mailbox, read/flag state, labels, and file index are replicated across devices as an **encrypted CRDT** synced over the mesh, converging under out-of-order delivery. Any device can send/receive; the always-on box is the anchor that guarantees receipt while other devices sleep.

5.7 Three products, shared components

Component	Mail	Chat	Files
Identity / keys (§1)			
MOTE object (§2)			
Mesh + mixnet (§4)			(control)
MLS group (§5.1)	(lists)	(channels)	(folders)
MLS KeyPackages (§5.3)			
Content-addressed blobs (§5.5)	(attach)	(attach)	
Fast tier (§4.6)	—	(live)	(bulk)
Device-cluster CRDT (§5.6)			

The one genuinely new component — the **MLS group** — unlocks all three. Real-time voice/video is **out of scope** (separate WebRTC/SFU architecture).

5.8 Groups as addressable identities

A group is an identity that has members. It has its **own keypair** and therefore its **own name** on the full naming ladder (§3.9): a group key-name, an `@handle` (e.g. `@team`), or a domain address (`team@company.com`). Sending to a group address delivers to all current members; membership is the group's MLS roster (§5.1). This single mechanism unifies mailing lists, chat channels, shared folders, and team inboxes — "a group with an address."

5.8.1 Two posting models

Model	Behavior
Broadcast / list	post to the address → every member receives a copy (distribution list, announce
Collaborative / channel	shared, ordered conversation + shared state (chat channel, shared folder)

Both are MLS groups; they differ only in **posting policy** and **membership-visibility policy**, which are fields of the group state. A group MAY switch models by policy change (a Commit).

5.8.2 Roles & management

Group state carries **roles**: `owner` (1), `admin`, `member`, and optional `poster` (may send) vs `reader`. Management operations are **MLS Commits** ordered by the group committer (§5.1), authorized by role:

- **Add / remove member** — Add uses the invitee's KeyPackage (§5.3) + Welcome; Remove triggers file-key rotation for shared folders (§6.7). Requires `admin`.
- **Role change / transfer ownership** — requires `admin` / `owner`.
- **Policy change** (posting model, membership visibility, join policy) — requires `admin`.
- **Join policy:** `closed` (invite only), `request` (request → admin approval), `open` (anyone with the address may join, rate-limited + anti-abuse §9), or `vouch` (a member introduces).

All membership/role/policy changes are signed and appear in the group's hash-chained handshake log (§5.1), so a member can audit "who added/removed whom" — the group analog of identity key-transparency (§3.5). A malicious/coerced committer can stall but not forge (every change is member-signed); forks are detectable.

5.8.3 Membership privacy (subscriber-list)

Broadcast lists MUST support **hidden membership**: members receive via per-member sealed delivery (§6) and do not learn the other members' keys. MLS's tree exposes members to *each other* by default, so hidden-membership lists use a **relay/committer fan-out** where the list identity re-seals to each member individually rather than a shared member-visible tree — at the cost of the shared-group efficiency. Channels (member-visible) use the normal MLS tree. This resolves the earlier mailing-list-privacy gap: choose the model per group, and disclose it.

5.8.4 Delivery & scale

- Small groups: standard MLS group message, fan-out to members over the mesh/mixnet.
- Large lists: the committer's ordered log is the source of truth; delivery is **per-member** (sealed to each), pull-or-push, so a 10k-member list is 10k individual sealed deliveries, not a 10k-member MLS tree. This trades cryptographic group-sharing for scalability and hidden membership — the right trade for announce lists.

5.8.5 Legacy interop

A group MAY have a **legacy address** (`team@company.com`) served by the gateway (§7): inbound legacy mail to the address is fanned out to members as MOTEs; outbound from the list to legacy subscribers goes via the gateway. So a DMTAP group is reachable as an ordinary mailing-list address from the old world, while native members get the full encrypted/group experience.

6. Privacy & Threat Model

DMTAP protects **content, authenticity, and metadata**. This section states the threat model honestly, defines the guarantees, and marks the boundaries we do *not* claim.

6.1 Adversary model

Adversary	Capability	DMT
Network eavesdropper (local)	reads links near a node	defea
Malicious relay / mix node	sees ciphertext it forwards	defea
Curious directory / DNS / KT log	sees lookups	mitig
Global passive adversary	observes all links, all timing	**prim**
Global *active* adversary (inject/drop/delay at will, unlimited resources)	shapes traffic to correlate	**not f**
Compromised endpoint (node seized/keylogged)	reads that node's plaintext	out o

DMTAP's headline guarantee is **strong metadata privacy against a global *passive* adversary.**

6.2 What is protected, and how

- **Content** — MLS/HPKE end-to-end encryption (§2, §5). Only recipients decrypt.
- **Authenticity** — signed payloads; identity bound to name via KT (§1, §3).
- **Sender identity** — **sealed sender**: the sender's identity and authenticating signature live *inside* the encrypted payload; intermediaries see only ciphertext to an opaque destination (§2.2). Honest scope: sealed sender hides the sender from *intermediaries*, but **does not hide the sender's IP** (the mixnet does that) and is a metadata-*reduction*, not elimination — receipt/timing side channels can statistically erode it (Martiny et al., NDSS 2021), which is why cover traffic (§6.3) and mixing are load-bearing, not optional.
- **Recipient identity** — the mixnet delivers by **push to an always-on node** whose network identity is decoupled from the human identity; there is **no store-and-poll step to hide**, which dissolves the private-retrieval (PIR) problem by architecture (§6.4).
- **Social graph & timing** — **mixnet** (onion routing + Poisson mixing delays) so no node sees both ends and timing correlation is defeated; plus **cover traffic** and **size padding** (§6.3).
- **Discovery** — name→key lookups routed through the mixnet, hiding *who* looks up *whom* (§3.7).

6.3 Mixnet, cover traffic, padding

- **Onion packets** (Sphinx-format, constant length) traverse a path of mix nodes; each peels one layer.
- **Mixing delay** — each mix holds packets a randomized (Poisson) time and reorders, so input and output streams cannot be timing-correlated.
- **Cover traffic** — nodes emit loop and drop cover messages at a steady rate; an observer cannot distinguish real activity from cover. Rate is a **tunable knob** (higher rate → more privacy, more bandwidth).
- **Size padding** — MOTEs are padded to fixed size buckets so length does not leak.

Email's asynchrony is the enabling property: minutes of latency are acceptable for the `private` tier, and higher latency yields stronger anonymity.

6.4 Why we avoid PIR (honest framing)

PIR (Pung, Talek) exists to hide **which record a client reads from an untrusted shared mailbox that holds everyone's mail** — a leak that only exists *because* those systems use a polled central store to support offline recipients. DMTAP makes a different **design choice**: the recipient runs an **always-on node that receives by push** through the mixnet, so there is no untrusted shared store being queried and thus no read-access-pattern to hide. We therefore avoid PIR's *problem* (and its large cost) — this is a genuine architectural difference, not a trick. (Note: Loopix itself *does* use provider store-and-poll to serve offline clients; DMTAP's always-on-push is a deliberate divergence, not an inherent property of mixnets.)

But push delivery is not "recipient anonymity, solved." Three residual exposures remain and MUST be handled:

1. **Last-hop / receipt visibility.** The final mix and a global observer of the recipient's link learn that *a packet was delivered to node X*. An always-on node has a **stable, targetable network presence** — itself a fingerprint. The node's network identity MUST be cryptographically and durably **decoupled** from the human identity (pseudonymous peer id, no re-identifying metadata).
2. **Receipt-timing.** Without cover on the delivery link, observing the node reveals *when* messages arrive. The recipient node MUST receive a steady **Poisson cover stream** so real receipts are indistinguishable from cover (as Loopix does with loop + link cover).
3. **Long-term intersection / statistical-disclosure attacks.** Persistent presence is exposed to correlation across many rounds; cover traffic bounds but does not eliminate this (cf. Vuvuzela's differential-privacy noise, which degrades over volume). This is inherited, not solved.

Offline buffering: if a node is down, a peer/relay holds sealed ciphertext, retrieved on return via an **unlinkable dead-drop token over the mixnet** (the buffer sees an anonymous pickup, not "user X"). Cheaper than PIR and sufficient for v0; full PIR remains an option for hostile-buffer scenarios.

6.5 Privacy tiers (and where each product sits)

Tier	Path	Latency	Graph privacy	Default for
`private`	full mixnet + cover	minutes	full (global passive)	mail, all control MOTEs, **normal-size fi**
`fast`	direct / few-hop	sub-second	content only	live chat (both online), **large-file bulk**

- Default is `private`. `fast` is opt-in. Because *choosing* `fast` is itself a signal, clients keep `private` as the standing default and cover it with cover traffic.
- **Files:** the control MOTE is always `private`. **Normal-size files route through the mixnet like messages (fully metadata-private). Large-file bulk** uses `fast` — but MUST be **onion-routed (a few hops, Tor-style) + size-padded to buckets + swarmed from multiple holders**, accepting bandwidth cost. **This bulk tier is explicitly weaker:** like Tor, it provides relationship anonymity against *local/partial* adversaries but is **vulnerable to end-to-end traffic correlation** by an adversary observing both endpoints (Murdoch–Danezis, 2005). Swarming and padding raise cost; they do not guarantee anonymity. The strong guarantee is the *messaging* tier; moving large sensitive files inherits Tor's correlation exposure. See §5.5, §4.5.

6.6 Honest boundaries (what we do NOT claim)

1. **Global active adversary.** An adversary who can inject, drop, and delay traffic everywhere and correlate at scale can, given enough resources, degrade anonymity. Perfect resistance is bounded by the fundamental **anonymity (latency + bandwidth)** tradeoff; more cover traffic and delay approach it asymptotically but never reach a costless zero.
2. **Large-file bulk metadata.** Onion-routing + padding + swarming makes it *strong*, not *free* and not *perfect* — the fact and approximate volume of a large transfer may remain partially observable at high adversary capability.
3. **Endpoint compromise has cluster-wide blast radius.** A seized/keylogged device that is a synced cluster member (§5.6) exposes the **entire replicated mailbox and its history**, not just a device-local slice — phone theft is the common case. Mitigated by at-rest encryption, forward secrecy (§5.2), recovery/rotation (§1), and OPTIONAL **scoped sync** (e.g. recent-N- days on mobile) which implementations SHOULD offer; not eliminated.
4. **First-contact MITM.** Before KT/OOB verification, the *first* name→key resolution can be MITM'd; DMTAP fails closed if KT is unreachable at first contact (§3.3, §3.4).
5. **Metadata privacy is weakest-link.** One leaky component (a bad lookup, a client bug, a timing side-channel) can unravel graph privacy. It is harder to *achieve* than to *design*; implementations MUST be conservative and audited.
6. **v0 key transparency is not equivocation-proof.** A v0 single, non-gossiped KT log can show different histories to different observers (split-view) until v1 gossip/multi-log auditing exists (§3.5). In v0, KT is **tamper-evident-after-the-fact and self-monitorable, but not a trusted single source** — a network SHOULD run multiple independent logs even in v0, and clients SHOULD treat a single unaudited log as advisory, leaning on OOB verification for high-value contacts. This is a real tension with the sovereignty goal and is stated as such.
7. **Group handshake ordering is a metadata concentration point.** The per-group committer/ ordering channel (§5.1) necessarily sees all of a group's handshake traffic; this is an explicit exception to the "no single node sees both ends" framing, bounded by rotating the committer and keeping application traffic on the mixnet.
8. `redact` / `expires` **are unenforceable** against a non-compliant recipient that already holds the plaintext — they are cooperative hints, not guarantees (relevant to any "right to erasure" framing).

DMTAP states these boundaries in-product. Honest, disclosed limits beat a false "perfectly anonymous."

6.7 Data at rest

- The node encrypts the mailbox, file blobs, and keys **at rest** under a device/identity key.
- Messages get MLS **forward secrecy**; files use **per-file keys** (blast radius = one file) delivered over the forward-secret channel. Persistent, re-readable data cannot ratchet like ephemeral messages — this is a property of files, not a DMTAP flaw (§5.5).
- **Access revocation on group-member removal (normative).** MLS removal blocks a removed member's *future* messages but does NOT revoke file keys they already hold. Therefore, when a member is removed from a shared file folder (§5.5), the node SHOULD (MUST for folders marked confidential) **re-key and re-encrypt** every file the removed member had access to, and redistribute the new keys to remaining members. Without this, a removed member retains indefinite read access to already-shared files. Clients MUST surface that pre-removal copies a member already downloaded cannot be recalled (the un-share limit, § file-safety).
- Recovery custody (§1.4) governs how at-rest keys survive device loss.

7. The Legacy Gateway (optional)

The gateway is the **only** component that speaks SMTP and the **only** one not content-blind (the legacy leg is unavoidably plaintext). It is **optional** — a node with no legacy correspondents never uses one, and at full DMTAP adoption it is unnecessary. It MAY be the node binary run in `--gateway` mode by an operator with a reputable IP and a domain.

7.1 Responsibilities

- **Inbound** (legacy → DMTAP): act as MX for a domain, receive SMTP, translate to a MOTE, attest it, and deliver into the mesh.
- **Outbound** (DMTAP → legacy): accept a `mail` MOTE marked for a legacy address, translate to RFC 5322, DKIM-sign as the sender's domain via **delegated selectors**, and send via SMTP.
- Carry the one irreducible operational cost: **IP reputation** (warmup, feedback loops, blocklist remediation, abuse handling).

The gateway is **stateless**: durability is punted to the edges (§7.4).

7.2 Inbound

1. Gmail connects to def.com MX = gateway; SMTP transaction.
2. Reject spam early, before DATA where possible (RBL/DNSBL, SPF/DMARC, greylisting, per-IP rate limits) — never accept the bulk of spam (§9).
3. Look up recipient key K for RCPT TO (via DNS/directory §3).
4. Wrap the RFC 5322 message into a MOTE (kind=mail), encrypt to K, and set an ATTESTATION: gateway signs "received via gateway G at T from <SMTP envelope>" **with a domain-anchored attestation key** (§7.2a), so the recipient can verify it arrived through a gateway genuinely authorized for the recipient's domain.
5. Deliver into the mesh to K (§4). If K's node is unreachable, return SMTP 4xx so the SENDING server retries (durability punted to the legacy sender). Store nothing.

7.2a Attestation key binding (normative)

An attestation is worthless unless its signing key is provably bound to a gateway the domain actually authorized — otherwise any operator could forge ”legitimate legacy origin” for a domain it does not serve. Therefore the domain MUST publish the gateway's **attestation public key**, analogous to DKIM's selector, in DNS (and MAY anchor it in KT):

```
<sel>._dmtap-gw.def.com.  IN  TXT  "v=dmtapgw1; k=<attestation public key>"
```

Recipient nodes **MUST** verify an inbound MOTE's attestation signature against a key published under the recipient's own domain (or an explicitly trusted gateway set), **MUST** reject attestations that do not verify, and **MUST** mark accepted ones as *legacy-origin* (not end-to-end encrypted before the gateway). This upgrades the former ”MAY verify” to a default-on check with a cryptographic anchor.

7.3 Outbound & DKIM delegation

1. node → gateway: a mail MOTE for bob@gmail.com (over the mesh, authenticated).
2. gateway translates MOTE → RFC 5322.
3. gateway DKIM-signs as the sender's domain using a DELEGATED selector: the domain owner publishes the gateway's DKIM public key at <selector>._domainkey.def.com —

so the

```
   gateway signs AS def.com WITHOUT ever holding the user's DMTAP identity key.
4. gateway SMTPs to the destination MX, enforcing TLS via MTA-STS/DANE.
5. On failure, report to the node; the NODE retries (owns the queue). Store nothing.
```

DKIM delegation cleanly separates *deliverability reputation* (the gateway's) from *identity* (the user's key, never shared).

7.4 Statelessness & durability

- Inbound: unreachable recipient → SMTP `4xx` → the **legacy sender** retries.
- Outbound: send failure → the **user's node** retries.
- The gateway holds no queue and no mailbox. Restart it and nodes/senders re-drive; nothing to recover or leak.

7.5 Decentralization & economics (summary; anti-abuse in §9)

- Many independent gateway operators MAY register in a directory with `{pubkey, reputation, region, price, stake}`; nodes select by reputation-to-destination.
- Per-identity accountability + operator stake keep a shared reputation pool safe to decentralize (§9). DKIM delegation makes operators swappable (change the delegated selector).
- Postage (§9) MAY fund outbound legacy sending (the stamp the sender attaches is redeemed by the delivering gateway), making it revenue-neutral and doubling as spam pricing.

7.6 Dual-stack addressing

A single `abc@def.com` is reachable both ways (§3.2): DMTAP senders resolve name→key and go native (mesh, no gateway); legacy senders use MX→gateway. Capability is discovered per-sender; no coordination is required, and native traffic grows while gateway traffic fades.

7.7 Fairness, self-host backstop & non-lock-in

DMTAP does not — and cannot — mandate that every gateway serve everyone. It instead makes **no gateway load-bearing**, so fair access is a structural property, not a rule requiring anyone's compliance.

You never need a specific gateway

- **DMTAP-native needs no gateway.** DMTAP DMTAP delivery is key-based over the mesh (§4); a user with only DMTAP correspondents never invokes one.
- **Self-host is always available.** Any user with a reputable IP and a domain MAY run the gateway (§7) themselves and depend on no third party. This **self-host backstop** makes gateway access a *right* (you can always serve yourself), not a grant.

Why a universal-service mandate is infeasible

A protocol rule ”every gateway MUST accept all traffic” is:

1. **Unenforceable** — no authority can compel a sovereign operator in a decentralized system; refusal or silent degradation cannot be prevented.
2. **Economically self-defeating** — a gateway forced to accept all traffic inherits the abuse that destroys its IP reputation, degrading the service for everyone. This is precisely why open SMTP relays died.

   So DMTAP does not attempt it. Fairness is achieved by the four mechanisms below instead.

How fairness is achieved

1. **The self-host backstop** (above) — the guaranteed right to serve yourself.
2. **A competitive, swappable market.** DKIM delegation (§7.3) + key-identity (§1) mean **zero lock-in**: a user switches operators with a DNS/DKIM change and no data migration (the box is the authority). If one operator refuses, another serves; switching is free.
3. **The accountability layer makes open service viable.** Open relays failed because abuse was unattributable. DMTAP attributes every send to an anonymous-but-accountable ARC token + optional postage + operator stake (§9), so a gateway *can afford* to serve strangers openly — abuse is priced and per-token-bannable, not a reputation-destroying free-for-all. Openness is no longer suicidal.
4. **An optional commons gateway.** A non-profit or protocol-funded operator MAY commit to universal, non-discriminatory service (a "public option"), funded by postage/donations, as **one operator among many** — not a mandate on all. Reputation ratings (§7.5) reward open operators and down-rank discriminatory ones, applying soft, market-driven fairness pressure.

The result is stronger than a mandate could enforce: because you can always route around, self-serve, or switch — and because accountability makes genuinely-open gateways survivable — no operator can act as a gatekeeper, without any unenforceable obligation being imposed.

8. Client Access

The node is a multi-protocol front-end over one MOTE store. Every protocol is a *view* of the same mailbox. Client-facing protocols terminate **on the node**; the node is reached through the mesh (SNI/stream routing over the relay/mesh) so no static IP is needed.

8.1 JMAP (native)

- The node exposes **JMAP** (RFC 8620 / RFC 8621) over a local HTTP endpoint (and, for remote devices, over an authenticated mesh stream).
- JMAP is the modern sync surface: `query`, `get`, `set`, `changes`, push. It maps directly onto the MOTE store and the device-cluster CRDT (§5.6).
- New DMTAP-native clients SHOULD prefer JMAP + the native MOTE/MLS APIs, which expose DMTAP-only features (identity verification, postage, privacy tier, file manifests).

8.2 IMAP / POP / SMTP-submission (compatibility)

To let existing mail clients (Apple Mail, Outlook, Thunderbird, mutt) work unchanged:

- The node runs **IMAP**, **POP3**, and **SMTP-submission** servers locally, projecting the MOTE store as folders/flags and accepting outbound submissions.
- Reached through the mesh (SNI-passthrough / stream routing); the node terminates TLS and speaks the legacy protocol; the relay/mesh never decrypts.
- **Auth = app-passwords**: the node issues app-specific passwords mapped to the identity, so legacy clients authenticate without touching the keypair.
- **Encryption is transparent to the owner's own client**: the node decrypts MOTEs and presents normal RFC 5322/MIME to the authenticated client. The E2E boundary is between *parties*, not between the owner and their own device.
- Outbound submission → the node converts to a MOTE (native) or routes to a gateway (legacy destination).

Compatibility support MAY be deprecated over time (like SMTP) as native clients mature; it is not required for conformance, but is RECOMMENDED for adoption.

8.3 Multi-device

Devices form the owner's personal MLS cluster (§5.6); each runs its own client surface and syncs the mailbox/flags/labels/file-index via encrypted CRDT over the mesh. The always-on node anchors receipt while other devices sleep.

8.4 Calendar & contacts (first-class, decentralized)

Calendar and contacts are **not** separate central services — they are additional **MOTE kinds** stored in the same node, end-to-end encrypted, synced across the device cluster (§5.6), and shared/invited via the same MLS groups (§5) as everything else. They inherit the full decentralized model: your node holds them, no provider can read them, and there is no central CalDAV/CardDAV server.

- **Native:** calendar events and contacts are represented as **JSCalendar (RFC 8984)** and **JSContact (RFC 9553)** MOTEs, synced via **JMAP** (Calendars/Contacts) alongside mail (§8.1). Calendar invitations and scheduling (iTIP-style) ride as MOTEs between participants — no central scheduling server; free/busy and RSVP are messages, not a server query.
- **Compatibility:** the node exposes **CalDAV (RFC 4791)** and **CardDAV (RFC 6352)** servers on `localhost` /over the mesh, projecting the calendar/contact MOTE store as iCalendar (RFC 5545) and vCard (RFC 6350) so existing clients (Apple Calendar/Contacts, Thunderbird, DAVx) work unchanged — reached through the mesh with app-passwords, exactly like IMAP (§8.2).

Compatibility DAV surfaces MAY be deprecated over time like the mail ones; native JMAP + MOTE is the forward path.

8.5 The decentralization invariant (all data classes)

Every data class DMTAP carries — **mail, chat, files, calendar, contacts, identity, and login (§13)** — obeys the same rule: it lives on the **user's node**, is **end-to-end encrypted**, syncs across the user's **device cluster**, shares via the same **MLS groups**, and routes over the same **mesh/mixnet** — with **no central server** for any of it. Legacy protocols (IMAP/POP/SMTP/CalDAV/CardDAV) and the OIDC bridge (§13.6) are **edge-compat surfaces only**; they never become a central store or a required intermediary. The node is the authority for *everything*, uniformly. There is no data class that quietly depends on a central service.

9. Anti-Abuse & Postage

DMTAP must stop spam/abuse **without** a central filter and **without** breaking sealed-sender anonymity. The tension: sealed sender hides who sent a message, but the recipient still needs to rate-limit and block abusers. The resolution is **anonymous but accountable** tokens.

9.1 Principles

1. **Authenticated-by-default.** Every native MOTE is authenticated to the recipient (payload signature, §2.4); there is no anonymous unauthenticated injection like open SMTP.
2. **Recipient policy is local.** The recipient node decides what to accept — allow known contacts, challenge strangers, block per-identity — *before* surfacing to the user (§2.7).
3. **Cost for cold contact.** Reaching a stranger costs something (a token, proof-of-work, or a paid stamp), so bulk unsolicited sending is uneconomic. Known contacts are free.
4. **Anonymity preserved.** Abuse control MUST NOT deanonymize the sender or link a sender across recipients.

9.2 Recipient policy

```
Policy {
  allow:      [* ContactRef],   // known keys / group members → free, always accept
  challenge:  ChallengeSpec,    // unknown senders must satisfy this
  block:      [* Token/KeyRef], // per-identity or per-token blocks
  rate:       RateLimit,        // per-sender-token limits
}
ChallengeSpec = pow(bits) / token(issuer) / stamp(amount) / vouch(guardianSet)
```

A stranger's first MOTE MUST carry a valid challenge response or it is dropped/deferred to a ”requests” area, never delivered to the inbox.

9.3 Anonymous rate-limit tokens (the core mechanism)

DMTAP uses **Privacy-Pass anonymous tokens** (RFC 9576 architecture, RFC 9577 HTTP scheme, RFC 9578 issuance) so a recipient can rate-limit/block a sender **without learning who they are** and **without the sender being linkable across recipients**.

CAUTION — pick the right variant. Vanilla Privacy Pass tokens are issuance-unlinkable but do **not**, alone, give the exact combination DMTAP needs: *per-recipient rate-limiting* AND *cross-recipient unlinkability*. That combination requires per-origin-scoped issuance — **Anonymous Rate-limited Credentials (ARC)**, the Privacy Pass WG work (`draft-yun-privacypass-arc`). DMTAP tokens MUST use ARC-style per-origin binding, not plain tokens, for the anonymity + accountability guarantee to hold simultaneously.

- A sender obtains blinded tokens from an **issuer** bound to a *rate budget*.
- The sender attaches a token to a cold MOTE; the recipient verifies it is valid and unspent, enforces per-token rate limits, and can **block a token/issuer** if it misbehaves — all without identity.
- Tokens are **unlinkable across recipients** (blind signatures), so token use does not build a cross-recipient graph.

9.3.1 Issuer trust is what gives a token value (normative)

A token is only as trustworthy as its **issuer**, and issuance MUST have a cost or scarcity or the whole ”cost for cold contact” model collapses (a spammer would self-issue unlimited free tokens). Therefore:

- The recipient's policy scores issuers (parallel to gateway reputation, §7.5). A token's granted rate budget is a function of **issuer trust at the recipient**, not of the token alone.
- A token from an **unknown/unvetted issuer (including the sender's own node) carries a default rate budget of ZERO** — i.e. it counts as *no token*, forcing fallback to PoW (§9.4) or postage (§9.5). Self-issuance thus buys anonymity but **no cost relief**.
- Trusted issuers (a recipient's own provider, a staked gateway, a vouched community issuer) make issuance costly/rate-limited on their side and stake reputation on not minting for abusers; a recipient extends budget to their tokens.

This is the load-bearing rule that makes ARC anti-abuse real rather than bypassable.

9.3.2 The unlinkability collective-defense tension (honest)

ARC's cross-recipient unlinkability is in **direct tension** with identifying a repeat spammer who hits many recipients: the very property that protects a legitimate sender's social graph also prevents recipients from cross-linking an abuser. DMTAP resolves this at the **issuer layer,**

not the recipient layer: repeat abuse is bounded by the issuer throttling/revoking its own issuance to a misbehaving client (who is *not* anonymous to its own issuer), and by recipients down-scoring issuers that mint for abusers. Recipient-side cross-recipient linking is deliberately unavailable; DMTAP does not claim it. (Signal solves the analogous problem with delivery tokens; DMTAP generalizes it to open, issuer-scored anonymous tokens.)

9.4 Proof-of-work (fallback)

Where no token issuer is available, a stranger MAY attach a **proof-of-work** (a hashcash-style puzzle over the MOTE `id` + recipient + date) whose difficulty the recipient policy sets. PoW imposes cost on bulk senders while remaining trivial for a single legitimate message.

CAUTION. PoW is a **last-resort** tier, not primary: there is no live standard, and plain hashcash asymmetry favors organized spammers (botnets/GPUs) over low-power legitimate senders (phones). DMTAP PoW MUST use a **memory-hard** function (Argon2/scrypt-style) to flatten that asymmetry, and difficulty SHOULD be adaptive. Prefer the token/reputation path (§9.3) whenever available.

9.5 Postage (economic tier + gateway funding)

Postage is a signed, prepaid, real-money credit voucher (NOT a cryptocurrency):

- A sender MAY attach postage to a cold MOTE; heavy senders pay, contacts are free.
- **Postage doubles as the gateway fee**: when a MOTE is bridged to legacy (§7), the delivering gateway **redeems the stamp**, so outbound legacy sending is paid-for and revenue-neutral.
- Postage is enforced by the recipient/gateway; it is orthogonal to token/PoW anti-spam and MAY be combined.

9.5.1 Issuance, redemption & double-spend (normative)

Postage is a **bearer instrument for real money**, so it needs a settlement model, not just a signature. DMTAP specifies the minimum:

- **Issuance:** a postage **issuer** (a provider/gateway with a real-money float) mints a stamp = `sign_issuer(serial, amount, expiry, [audience])`. The issuer debits the buyer's account at mint time. `audience` MAY scope a stamp to a specific recipient/gateway.
- **Redemption + double-spend prevention:** a stamp is single-use. The **redeeming party (recipient node or gateway) MUST check the stamp's `serial` against the issuer's redemption endpoint** (online check) or accept the issuer's signed short-lived spent-list; the issuer marks the serial spent atomically. A stamp presented twice is rejected on the second redeem.
- **Cross-operator settlement:** the redeemer claims the amount from the issuer against the serial; issuers reconcile out-of-band (standard interchange). "Trivial" was an overstatement — it requires an issuer-side ledger, specified here.
- **Failure mode:** if the issuer is unreachable at redemption, the stamp is treated as *unverified* (falls back to token/PoW policy), never accepted on faith. No offline bearer acceptance of real money.

Postage issuers are subject to the same reputation/trust scoring as token issuers (§9.3.1); an untrusted issuer's stamps carry no weight.

9.6 Gateway-operator accountability

For decentralized legacy gateways (§7):

- **Per-identity accountability**: every outbound handoff carries the sender's signature + postage, so a gateway attributes abuse to a *token/identity*, not an IP — making a shared reputation pool safe to decentralize.
- **Operator stake/bond**: operators post collateral; poisoning the pool slashes it (Sybil resistance + incentive alignment).
- **Reputation routing**: nodes route outbound to operators by measured deliverability; bad operators lose traffic.

9.7 Vouch / introduction (optional)

A cold sender MAY present a **vouch** — an introduction token signed by a contact the recipient trusts (a mutual connection) — to bypass PoW/stamp. This bootstraps first contact through the social graph without a central authority, and MUST itself be rate-limited to prevent vouch farming.

9.8 Mixnet abuse

Mix nodes are content-blind and cannot filter content. Abuse of the mixnet itself (flooding) is bounded by: token/PoW admission at the *entry* (a MOTE without valid postage/token is not accepted into the mixnet by the sender's own entry policy), operator rate limits, and stake for mix operators (§6.4, §7.5). Cover traffic is rate-bounded per node.

10. Versioning, Conformance, Governance

10.1 Protocol versioning

- Every wire object carries a format version (`v`) and algorithm suite (`suite`).
- Unknown `v` / `suite` MUST be rejected (fail closed), never guessed.
- New message kinds use the reserved `0x40–0x7f` range (§2.3); a node ignores unknown kinds it is not required to process, but MUST NOT ack a kind it cannot validate.

10.2 Capability negotiation (dual-stack)

- A sender discovers a recipient's capabilities via the recipient's `Identity` /DNS record (native DMTAP vs legacy-only) and picks the path per-recipient (§3, §7.6).
- `system` MOTEs (kind `0x0a`) carry capability announcements between nodes (supported suites, privacy tiers, MLS ciphersuites, extensions).
- **Opportunistic upgrade**: a legacy correspondent is reached via the gateway; if they later publish a DMTAP key, native delivery is used automatically. No flag day.

10.3 Conformance levels

An implementation is **DMTAP-conformant** at a level if it passes the corresponding suite:

Level	Requires
Core	Identity (§1), MOTE (§2), naming resolution + TOFU + **v0-minimal KT publish/ver**
Private	Core + mixnet + sealed sender + cover traffic + privacy tiers (§4, §6)
Groups & Files	Core + MLS groups + content-addressed file transfer (§5)
Legacy	Core + gateway inbound/outbound + DKIM delegation (§7)
Clients	Core + JMAP; IMAP/POP/SMTP-submission compat RECOMMENDED (§8)
Auth	Core + DMTAP-Auth login ceremony with origin binding + key-bound sessions (§13); O

The **conformance test suite** (in `conformance/`, TBD) is the *operational definition* of compatibility. "DMTAP-compatible" means "passes the suite," not "resembles the reference." This is the primary defense against fragmentation.

10.4 The spec is authoritative

Independent implementations MUST be buildable from this specification alone. The Rust reference in `node/` and `gateway/` is a proof and a set of libraries, **not** normative. Where reference and spec disagree, the spec governs (or the discrepancy is filed as a bug).

10.5 Governance & licensing (intent)

- **Standards track:** pursue an **IETF Internet-Draft** for the wire protocol and object formats, aiming for RFC status (as JMAP and MLS did). Neutral governance is what lets competitors adopt without fearing capture.
- **Licensing:** the **specification** and the **reference implementation** (node, gateway, client, libraries) ship under the **MIT license** (Apache-2.0 dual-licensing under consideration for its explicit patent grant). Everything a user touches and everything trust depends on is open; permissive licensing maximizes adoption by any party, competitors included.
- **Open software + paid operations:** commercial sustainability comes from a thin, **private control-plane** (a hosted operator) that implements the **operator seam** (`crates/dmtap-seam`) to bill *operations* — never gating any protocol or privacy feature. See §12 for the full model and the inviolable rule (privacy/crypto are never behind the seam).
- **Reuse over reinvention:** DMTAP deliberately composes existing standards — MLS (RFC 9420), HPKE (RFC 9180), JMAP (RFC 8620/8621), libp2p, DNS/DNSSEC/SVCB, key transparency, Privacy Pass. The novelty is the *composition and transport*, not new cryptography.

10.6 Roadmap markers (non-normative)

- **v0:** Core + Private (minimal KT via TOFU+pinning) + Groups & Files + Legacy gateway.
- **v1 hardening:** full CONIKS/KT with monitoring + gossip; onion-routed bulk; anonymous tokens at scale; PQ suite `0x02` migration; optional self-sovereign naming backend.
- **Later research:** stronger private contact discovery; scalable private retrieval for hostile-buffer scenarios; deniable-group properties; metadata-privacy for very large files.

11. Grounding & References

This section records the standards each design choice rests on, the corrections applied after verification, and a consolidated statement of honest limits. DMTAP deliberately **composes existing standards**; the novelty is the composition and transport, not new cryptography.

11.1 Verified building blocks

Layer	Choice	Standard / sourc
PKE	HPKE	RFC 9180 (DHK
Signatures	Ed25519 (PureEdDSA)	RFC 8032
Key agreement	X25519	RFC 7748
PQ KEM	ML-KEM-768 (cat-3)	FIPS 203
PQ signatures	ML-DSA-65 (cat-3)	FIPS 204 (SLH-I
Hybrid HPKE KEM	**X-Wing** (ML-KEM-768 X25519)	`draft-connolly-c`

Layer	Choice	Standard / sourc
Hashing	BLAKE3-256 + agility prefix	(BLAKE3 not F
Serialization	Deterministic CBOR (bytewise/core key order)	RFC 8949 §4.2 (
Group/session	MLS	RFC 9420; archi
PQ-MLS	ML-KEM ciphersuites + combiner	`draft-ietf-mls-p`
Async join	MLS KeyPackages + external commits	RFC 9420
Sealed sender	payload-embedded sender	Signal (2018)
Mixnet packet	Sphinx	Danezis & Goldb
Mixnet design	Loopix (Poisson mix, loop/drop/mix cover)	Piotrowska et al
Client sync	JMAP	RFC 8620 / RFC
Mesh	libp2p (Kad, Relay v2, DCUtR, AutoNAT v2, Noise, QUIC)	libp2p specs
Location record	IPNS-style signed value record	IPFS/IPNS
Naming	DNS TXT (key) + SVCB (service)	RFC 9460; cf. D
Key transparency	Merkle log + STH + inclusion/consistency/absence	CT RFC 6962; C
Anti-abuse tokens	Privacy Pass + **ARC** scoping	RFC 9576/9577/
PoW fallback	memory-hard hashcash	Back 1997; Argo
Recovery	VSS + SLIP-0039 + **FROST**	Feldman/Pederse
Self-sovereign naming (opt)	name-chain (ENS-style)	Zooko's Triangle

11.2 Corrections applied after verification

1. **MLS handshake ordering** — Commit/Proposal/Welcome MUST use an ordered channel, **not** the reordering mixnet (§5.1). This is the hardest part of MLS-over-mesh; track `draft-kohbrok-mls-dmls`.
2. **Async init** — use MLS-native **KeyPackages + external commits**, not a bolted-on PQXDH (§5.3).
3. **Deniability** — MLS is **non-repudiable**; DMTAP does **not** claim message deniability and does not swap MLS signatures for MACs (§5.2). Open design item.
4. **No-PIR claim** — reframed honestly: push delivery avoids PIR's problem but leaves last-hop visibility, receipt-timing, and intersection exposure — mitigated by node/identity decoupling + recipient-side cover traffic (§6.4).
5. **DHT security** — record signing eclipse resistance; added S/Kademlia disjoint paths, IP-diversity buckets, seq#/EOL rollback defense, and a **non-DHT rendezvous fallback**; the DHT is one discovery mechanism, not the root of trust (§4.2).
6. **QUIC roaming** — carried by location re-publish + re-dial, not seamless QUIC migration (`quinn` has no multipath) (§4.1).
7. **Sealed sender scope** — hides sender from intermediaries, not the IP (mixnet does that); metadata-*reduction*, not elimination (NDSS 2021 side-channel) (§6.2).
8. **Bulk-file tier** — explicitly weaker; Tor-style onion routing is subject to end-to-end correlation; swarming/padding raise cost, not guarantees (§6.5).
9. **Recovery** — VSS over plain Shamir; FROST to avoid key reassembly; SLIP-0039 encoding; ”redistribution” vs ”proactive refresh” distinguished (§1.4).
10. **Anti-abuse tokens** — need **ARC** (per-origin rate-limited + cross-recipient unlinkable), not vanilla Privacy Pass; PoW must be **memory-hard**, last-resort (§9).
11. **Hash agility** — content-address digests carry a multihash-style prefix (§2.2).
12. **DNS is discovery, not trust** — a DNS binding is only trustworthy via KT; SVCB is for service params, a dedicated TXT for the key (§3).
13. **DTN / radio** — store-and-forward is a DMTAP-built layer (not libp2p); Bluetooth/Wi-Fi Direct/LoRa are future work, not v0 (§4.8).

11.3 Consolidated honest limits

- **Global active adversary** with unlimited resources is not fully defeated — the **Anonymity Trilemma** (Das et al., IEEE S&P 2018) makes strong anonymity cost latency and/or bandwidth. DMTAP targets a **global passive adversary** and *bounds* (not eliminates) partial-active exposure.
- **Intersection / statistical-disclosure attacks** against always-on nodes are inherited; cover traffic bounds, does not eliminate, them.
- **First-contact MITM** is possible until KT gossip or out-of-band verification (§3.4).
- **KT without gossip** catches later tampering and enables self-monitoring, but a single equivocating log is not caught until independent auditors/gossip exist (§3.5).
- **Large-file bulk** metadata is weaker than message metadata (Tor-class), by necessity.
- **Persistent-file forward secrecy** cannot match ephemeral-message FS (files must stay readable); mitigated by per-file keys + FS-delivered keys + at-rest encryption (§6.7).
- **Key loss** of `IK` plus all recovery factors is unrecoverable (the bottom turtle, §1.4).
- **PQ onion (Sphinx)** as published is not PQ-safe; PQ mix packet formats are open research.

11.4 Implementation-stack notes (reference, Rust)

- **Prefer formally-verified PQ crates** (`libcrux-ml-kem` , `libcrux-ml-dsa`) over the unaudited RustCrypto `ml-kem` / `ml-dsa` ; pin past the `ml-dsa` advisory GHSA-5x2r-hc65-25f9.
- **openmls** — independently audited (SRLabs, 2025) — is the MLS choice.
- **hpke** (rozbb) or **hpke-rs** (Cryspen) — not `ed25519-dalek-hpke` (experimental).
- **ed25519-dalek 2.x**, **x25519-dalek**, **blake3**, **ciborium** — mature.
- **Mixnet**: `sphinx-packet` / `nym-sphinx-*` are low-cadence standalone; track Nym directly.
- **rust-libp2p** is production-usable but churns its API (pin versions); go-libp2p is the more mature interop reference — a deliberate memory-safety-vs-maturity trade.

11.5 Selected bibliography

- Danezis & Goldberg, *Sphinx: A Compact and Provably Secure Mix Format*, IEEE S&P 2009.
- Piotrowska, Hayes, Elahi, Meiser, Danezis, *The Loopix Anonymity System*, USENIX Sec 2017.
- Das, Meiser, Mohammadi, Kate, *Anonymity Trilemma*, IEEE S&P 2018.
- van den Hooff et al., *Vuvuzela*, SOSP 2015; Angel & Setty, *Pung*, OSDI 2016; Cheng et al., *Talek*, ACSAC 2020.
- Dingledine, Mathewson, Syverson, *Tor*, USENIX Sec 2004; Murdoch & Danezis, *Low-Cost Traffic Analysis of Tor*, IEEE S&P 2005.
- Melara et al., *CONIKS*, USENIX Sec 2015; RFC 6962 (Certificate Transparency).
- Martiny et al., *Improving Signal's Sealed Sender*, NDSS 2021.
- RFCs 9180, 8032, 7748, 8949, 9420, 9750, 8620, 8621, 9460, 6376, 7929, 9576/9577/9578, 9591.
- FIPS 203, 204, 205; `draft-connolly-cfrg-xwing-kem` ; `draft-ietf-mls-pq-ciphersuites` ; `draft-ietf-mls-combiner` ; `draft-kohbrok-mls-dmls` ; `draft-yun-privacypass-arc` .

12. Operators, the Seam & Sustainability

DMTAP is deployed in one of two modes, and the boundary between them is a small, explicit **operator seam**. This section is partly *informative* (business/licensing intent) and partly *normative* (the inviolable rule and the seam's guarantees).

12.1 Two deployment modes

Mode	Who runs it	Cost	Limits
Self-host	The user, on their own box	$0	none — fully functional, unrestric
Hosted operator	A provider (e.g. a commercial control-plane)	paid	plan quotas on *operations* only

The OSS is a **complete product on its own**. Self-host is not a crippled tier: it has every protocol, client, and privacy feature. A hosted operator adds *convenience* (someone else runs the box, warms the IPs, manages the domain), not *capability*.

12.2 The operator seam

The seam is the clean boundary the OSS exposes so an operator can add billing and multi-tenant management **without forking or patching the protocol**. It is four capabilities (reference crate: `crates/dmtap-seam`, contract: its `CONTRACT.md`):

- **Metering** — the OSS emits usage events at the real cost centers (gateway egress, storage, relay bytes, message counts, vanity domains).
- **Provisioning** — create/suspend accounts and `@`-addresses across onboarding tiers A/B/C (§3.8).
- **Policy** — per-account quotas/entitlements on operations (storage caps, send caps, domain counts, rate limits).
- **GatewayAuthz** — authorize legacy egress with per-identity-token accountability (§9), preserving sealed sender.

Each capability has a **self-host default that is unlimited/no-op**, so with no operator the OSS runs unrestricted. A hosted operator implements the same capabilities — in-process (Rust traits) or out-of-process (HTTP/events) — to add billing and quotas.

Fail-open to function, fail-safe on security (MUST): if the operator is unreachable, the OSS MUST NOT break user-facing mail/chat/files. Metering queues and retries (usage may under-count during an outage), and quota **Policy** falls back to allow (a billing concern, not a security one). **GatewayAuthz is different and MUST NOT fail open to "allow."** GatewayAuthz is a security control (§9): the accountability it enforces is exactly what prevents unattributable legacy egress — the open-relay failure mode §7.7 exists to avoid. On operator-unreachable, GatewayAuthz MUST fall back to a **safe default**: permit legacy egress only to **already-established contacts** (and to senders carrying valid self-contained proof — postage/PoW verifiable without the operator), and **deny cold/unproven legacy egress** for the outage window. A generic "fail-open to allow" for this capability is prohibited.

12.3 The inviolable rule (normative)

Privacy, cryptography, metadata privacy, and recovery MUST NEVER be behind the seam or a paywall. There MUST be no seam hook, quota, or plan gate capable of disabling encryption, weakening the mixnet, reducing metadata privacy, or denying a user access to their own keys or mailbox. The seam meters and limits **operations and organizational concerns only**. Premium is for *running it*, never for *protection*.

A conformant operator implementation MUST NOT expose any control that violates this rule; a conformant OSS build MUST NOT consult the seam for any privacy/crypto decision.

12.4 Business & licensing model (informative)

DMTAP follows an **open-software + paid-operations** model — the GitLab-style split done cleanly, without a crippled free tier:

- **Everything a user touches, and everything trust depends on, is open** — protocol, spec, node, gateway, client, crypto. Shipped under the **MIT license** (Apache-2.0 dual-licensing is under consideration for its explicit patent grant, relevant to novel mechanisms such as anti-abuse postage/tokens). Libraries other implementations embed are permissively licensed to maximize adoption.
- **The paid layer is a thin, private control-plane** (a *hosted operator*) that implements the seam contract and bills **operations**: hosted nodes, storage, legacy egress / IP reputation, vanity domains, org/SLA. It withholds no protocol, client, or privacy feature.
- **The moat is reputation + network + being the best operator**, not the code. Because openness is what lets the network grow large enough to be worth hosting, full openness costs little and is itself a trust requirement for a privacy product (a closed client cannot credibly claim "we can't read your mail"). Competitors hosting DMTAP grow the network the operator is central to.

12.5 Honest limits

- **Free-rider economics:** paying (convenience) customers subsidize free self-hosters. Intended and viable (the convenience majority pays; the sovereign minority self-hosts), provided the paid tier genuinely covers cost + margin.
- **Commoditization:** open software means thin margins on the software itself; margin comes from operations and scale, not licenses.
- **No protocol control:** required for adoption, but it means an operator can be out-executed — the defense is execution and trust, not a license.
- **The open-core temptation recurs:** pressure to move "just one" feature behind the paywall will appear. The inviolable rule (§12.3) is the bright line; drift there is fatal to the brand and is prohibited for privacy/crypto features.

13. DMTAP-Auth — Decentralized Identity & Login

Your DMTAP identity is a keypair (§1) with a human name resolvable to that key (§3). The same identity that receives your mail can log you in **everywhere**, with **no central identity provider**. This section specifies DMTAP-Auth: sovereign, decentralized web login built on the identity and naming layers, plus a bridge so existing "Sign in with OIDC" apps work unchanged.

The governing principle mirrors the rest of DMTAP:

> **Your key is your identity — for mail, for messaging, and for login.** No provider can revoke it, surveil it, or lock you out. Your node is your identity provider.

13.1 Goals & non-goals

DMTAP-Auth MUST provide:

1. **Decentralized login** — a relying party (RP) authenticates you as `alice@yourdomain` by verifying a signature from your key; there is no Google/Okta in the middle.
2. **Phishing resistance** — a signature obtained by a look-alike site MUST NOT authenticate to the real site.
3. **No bearer tokens after login** — sessions are bound to your key (proof-of-possession), so a stolen token is useless without the key.
4. **Legacy interoperability** — existing OIDC/OAuth relying parties can consume DMTAP-Auth through a compatibility bridge.
5. **Recoverable & revocable** — losing a device does not lose your identity (§1.4), and a single app/device session can be revoked without rotating your whole identity.

Non-goals: replacing WebAuthn/passkeys (DMTAP-Auth *uses* them); acting as an authorization server for arbitrary third-party API scopes beyond login + basic profile + delegated capabilities (§13.5).

13.2 Substrate reuse

DMTAP-Auth introduces almost no new cryptography — it reuses:

- **Identity** (§1): root key `IK`, device subkeys, the signed `Identity` object.
- **Naming** (§3): `name → key` via DNS + key transparency — this *is* issuer discovery.
- **Recovery** (§1.4): because your key now guards *all* your logins, recovery is even more load-bearing; DMTAP-Auth inherits it directly.
- **Transparency log** (§3.5): device/session authorizations and revocations are logged, so unauthorized login grants are detectable (§13.4).

13.3 The native login ceremony

```
1. RP shows "Sign in with DMTAP"; user enters  alice@yourdomain
2. RP resolves name → key + auth endpoint      (§3 lookup; DID/OIDC discovery, §13.6)
3. RP creates a Challenge:
     { rp_origin, nonce, issued_at, exp, aud, [scope] }
4. The challenge is presented to a TRUSTED CLIENT on the user's side (browser/OS/app), which
   binds and displays the VERIFIED rp_origin, and runs a WebAuthn/passkey user-verification
   ceremony (§13.3.1)
5. The user's key signs  H(rp_origin ∥ nonce ∥ issued_at ∥ exp ∥ aud)  (canonical, §2)
6. RP verifies the signature against alice's pinned key (§3.4) and that rp_origin == its own
   origin, nonce unused, not expired → authenticated
```

The signed statement is a structured, origin-scoped, nonce-bound challenge (the SIWE/CAIP-122 pattern, hardened — see §13.7). The `aud` field binds the assertion to the intended RP.

13.3.1 Origin binding is the load-bearing property (normative)

Phishing resistance comes **entirely** from binding the assertion to the *true* RP origin, and that binding MUST be injected and enforced by a **trusted client component on the user's side**, never by the signer trusting a value handed to it by the RP.

- In a browser, this is **WebAuthn** (W3C Rec L2 / CR L3): the browser writes the *observed* origin into `clientDataJSON`, the authenticator signs over its hash, and credentials are scoped by `rpId`. An assertion produced at `alice-yourdomain.evil.com` cannot validate for `yourdomain`. DMTAP-Auth's browser ceremony MUST use WebAuthn for exactly this.
- **The remote-node hazard (CRITICAL).** If the user's *always-on node* signs challenges that arbitrary parties relay to it, origin binding **evaporates** — a phisher relays the real challenge, gets the node to sign, and replays it. FIDO2 cross-device auth (CTAP 2.2 "hybrid") mitigates the analogous remote-approval risk with **BLE proximity**, which a remote node cannot use. Therefore DMTAP-Auth REQUIRES, for any node-signed login:

  1. the challenge carries `rp_origin` and `aud`, and the node signs *over them*;
  2. a trusted approval surface (the node's own authenticated client/app, or a paired passkey) **displays the verified** `rp_origin` to the human and requires explicit approval per login;
  3. the node MUST reject a challenge whose `rp_origin` it cannot attribute to an authenticated request channel, and MUST rate-limit and log approvals (consent-farming defense).

Preferred design: the **passkey/WebAuthn ceremony happens in the user's client**, and the node's key is only invoked *after* a successful local user-verification bound to the origin — i.e. the passkey gates the node's signature. Concretely, use the **WebAuthn PRF extension** (over CTAP2 `hmac-secret`) so the passkey deterministically derives the key that unlocks the node's signing key — the identity key never leaves the node and never touches the RP (the deployed wwWallet pattern). Nodes MUST NOT offer "approve any challenge" modes.

13.4 Sessions: key-bound, not bearer

After login, the RP and client establish a session that is **sender-constrained to the user's key**, so a leaked token cannot be replayed by a thief:

- Sessions SHOULD use **DPoP (RFC 9449)** — each request carries a fresh proof-of-possession JWT signed by a session key bound at login — or **GNAP (RFC 9635)** continuation, which is key-based end to end.
- Session keys are **per-RP, per-device** ephemeral keys authorized by a device key (§1.2), not `IK` itself. This limits blast radius and enables granular revocation.
- **Revocation:** revoking one app/device session publishes a revocation to the transparency log and/or a short-lived status endpoint; it MUST NOT require rotating `IK`. Rotating a device key (§1.5) revokes all its sessions at once. Losing `IK` and recovering (§1.4) MUST invalidate all prior session authorizations.

13.5 Capabilities & delegation

For "let this app act on my behalf" (beyond login), DMTAP-Auth uses **capability delegation** rather than opaque scopes: a signed, offline-verifiable, attenuable capability token (**UCAN-style**, informative) delegates a *specific, least-privilege* right (e.g. "read calendar MOTEs for 24h") from `IK`/device key to an app or another of the user's nodes. Delegations are attenuable (a device can only sub-delegate a subset), time-bound, and revocable. This also carries authorization **across the user's own device cluster** (§5.6).

13.6 Legacy bridge: OIDC / OAuth compatibility

Existing apps speak "Sign in with Google/OIDC," not DMTAP-Auth. The bridge makes DMTAP identity consumable by them — the same native-path/legacy-bridge duality as mail (§7):

- **Per-user issuer (native-ish).** OpenID Connect Discovery 1.0 (Final) permits WebFinger-based **issuer discovery** from a user identifier, with the issuer at any host. So `alice@yourdomain` can advertise her **own node as her OIDC issuer**, self-issuing ID Tokens (the **SIOP v2** shape — a draft, not final). Deployed precedent: **Solid-OIDC** (user-controlled OPs; note it trusts a WebID-named *issuer*, it does not embed keys) and **IndieAuth** (identity = a URL you own; a W3C Note + IndieWeb living standard, not a Recommendation — cite the living standard). DMTAP-Auth expresses the binding as **`did:web` rooted at the user's mail domain** (`did:web:yourdomain:users:alice` → `did.json`), the DID method that matches §3's DNS name→key.
- **Hosted bridge (compat).** Because mainstream RP libraries assume a *fixed* issuer allowlist and rarely implement WebFinger discovery or dynamic client registration, DMTAP-Auth defines a **bridge OIDC Provider**: a standard, well-known OP whose backend is DMTAP-Auth. Legacy RPs add it like any OIDC provider; the bridge performs the §13.3 ceremony and mints a standard ID Token. The bridge is a *convenience operator*, swappable and self-hostable — it sees login events but never the user's key (it verifies signatures), and it is content-blind to mail.

- **DID interop.** `alice@yourdomain`'s `name→key` binding is expressible as **`did:web`** (DNS/HTTPS-rooted, matching §3) so DID-aware verifiers and Verifiable-Credential ecosystems (OID4VP, Final) can consume DMTAP identities. `did:key` expresses a raw-key identity (tier A, §3.6).

Adoption grows exactly like mail: native RPs verify signatures directly; legacy RPs use the bridge; the bridge's importance fades as native support spreads.

13.7 Grounding & honest limits

Grounded standards (verified):

Layer	Standard	Status
Origin-bound ceremony	WebAuthn (W3C Rec L2, CR L3) + CTAP2 (FIDO2)	Rec / final
Challenge pattern	SIWE **ERC-4361** (Final) + **CAIP-122**	final
Keypair web-auth precedent	Nostr **NIP-07 / NIP-98**	deployed
Key-bound sessions	**DPoP RFC 9449** / **GNAP RFC 9635**	RFC
Capability delegation	**UCAN v1.0**	community spec (inf
Issuer discovery / self-issued	**OIDC Discovery 1.0** (Final) + **SIOP v2** (draft)	final / draft
User-controlled OP precedent	**Solid-OIDC**, **IndieAuth** (living std)	deployed
Identifier interop	**DID Core** (W3C Rec), `did:web` / `did:key`	Rec / community
Modern OAuth security	**OAuth 2.1** (draft) + **RFC 9700** BCP	draft / RFC

Honest limits (stated in-product):

1. **Origin binding depends on a trusted client.** The anti-phishing guarantee holds only where a trusted browser/OS/app enforces `rp_origin` (WebAuthn). Raw keypair signing where the *user visually verifies* the origin (SIWE/NIP-98 style) is **weaker** and MUST NOT be the only mode; DMTAP-Auth improves on it via §13.3.1 but a compromised or absent trusted client degrades to user-verified security.
2. **Remote-node consent farming.** A remote always-on node approving relayed challenges cannot use proximity; the mitigations in §13.3.1 (origin display, per-login approval, channel attribution, rate-limit, log) bound but do not eliminate the risk. Passkey-gated signing is the safer default.
3. **Key loss = login loss.** Your key now guards every login, so recovery UX (§1.4) is critical; this raises the stakes on getting recovery right, not a new vulnerability.
4. **RP adoption is chicken-and-egg.** The OIDC bridge (§13.6) is what makes DMTAP-Auth useful before native RP support exists — the same bootstrap strategy as the mail gateway.
5. **SIOP v2 is a draft** and WebFinger issuer discovery is the least-implemented corner of OIDC; the hosted bridge is required precisely for this reason, not optional polish.
6. **No PKI beyond DNS/CA.** `did:web` (and DNS-rooted naming, §3) bottoms out at DNS + TLS/CA — there is no independent proof binding domain to key, so a DNS/registrar/CA compromise is an identity compromise. Key transparency (§3.5) makes such a substitution *detectable*, not impossible; high-value use SHOULD add out-of-band verification.
7. **Login is a deliberate identity disclosure.** Authenticating to a relying party intentionally reveals your identity *to that RP*; this is opt-in and per-RP, and MUST NOT be conflated with or allowed to weaken the mail/messaging metadata-privacy guarantees (§6). Session keys are per-RP (§13.4) so RPs cannot correlate you across sites via the auth layer.

14. Scaling & Deployment

DMTAP must work for a user with only a phone, a user with an always-on box, and an operator hosting millions of mailboxes — and the gateway fleet must scale horizontally. This section specifies the node classes, the reachability/buffer layers, and the scaling patterns, grounded in how production mail and P2P systems actually operate.

14.1 Node classes

Class	Examples	Role
Always-on node	Pi, NAS, home server, VPS	The authority — holds the mailbox, does
Intermittent node / thin client	Laptop, phone	A client that syncs to an always-on node
Relay	any public-IP node	Reachability hop for NAT'd nodes; conter
Mix	staked operator node	A mixnet hop (§6); content-blind
Gateway	accountable operator	Legacy SMTP bridge (§7); the only non-c

A phone is **never** a durable endpoint (§14.3). An always-on node is the durable authority (§14.4). Relay/mix/gateway are stateless middle roles that scale horizontally (§14.2, §14.5).

14.2 Horizontally-scalable gateways (normative patterns)

The gateway (§7) is stateless and MUST scale as a **shared-nothing worker fleet**:

1. **Identical, interchangeable instances.** Every gateway instance runs the same config; there is no whole-cluster coordination on the hot path (coordination is a scaling bottleneck — the KumoMTA design principle). Add instances to add capacity.
2. **IPs owned by an egress layer, not the instances.** Outbound source IPs are selected by an egress-proxy layer (weighted round-robin over **egress pools** of warmed IPs), so worker instances stay stateless and IPs are managed independently of instance count.
3. **DKIM keys distributed read-only** to all instances (never generated per-instance), so signing is stateless-safe.
4. **Per-ISP warmup state.** An IP is "warm for Gmail, cold for Outlook" independently; pool sizing and warmup track **per receiving ISP** (the SES managed-pool model), spilling overflow to a shared pool during warmup.
5. **One coordinating tier for the irreducible shared state.** IP reputation and rate/throttle counters are the *one* thing a shared-nothing fleet cannot make node-local; hold them in a small shared store (Redis-style) feeding automated **health-check pool demotion** (Good/Poor/New by bounce & complaint rates), not in the workers.
6. **Inbound MX at scale:** DNS MX priority + equal-preference randomization (RFC 5321 §5.1) for coarse load spreading, Anycast receiving IPs, and accept-then-forward MTAs behind a source-IP-preserving LB (XCLIENT/PROXY protocol) so downstream filtering still sees the true sender.

Honest limits (MUST disclose): IP reputation is irreducibly shared mutable state — the one non-local part of an otherwise shared-nothing fleet. **Warm-IP inventory is a hard scaling floor:** you cannot grow outbound faster than you can warm IPs (weeks), so burst capacity is bounded by the pre-warmed pool, not by instance count. DNS MX round-robin is sender-controlled (you cannot force even distribution). Anycast inbound must be paired with stable routing so a route flap does not break a live SMTP session.

14.3 The mobile-only user (no always-on box)

A phone cannot hold a socket open or be reachable, and platform push is **best-effort, not guaranteed** on both APNs and FCM. Therefore a phone is a **push-woken thin client that drains a queue it does not host** — never a durable node. When the user has no always-on box, the queue is a **hosted, content-blind relay-mailbox** (the Chatmail model):

```
sender → relay-mailbox (E2EE ciphertext, short TTL) → wake-push (content-free, ≤4KB) via
         APNs/FCM through a notification proxy → phone wakes, opens its own authenticated
         connection, DRAINS the queue, decrypts locally, then reconciles on foreground
```

Requirements:

- **Push carries no content** — only a wake signal (the device token is itself encrypted where possible). Apple/Google never see plaintext. The design is **wake-and-fetch**, never deliver-in-push (APNs payload 4KB; silent push is throttled if excessive).
- **Push is a latency optimization, not delivery.** The client MUST still poll/reconcile on foreground; DMTAP MUST NOT treat a push as delivery confirmation. Wakes SHOULD be coalesced/batched to avoid iOS silent-push throttling.
- **The relay-mailbox is a buffer, not an archive** (§14.5): short TTL (~weeks), content-blind, delete-after-inactivity. The durable copy lands on the device once fetched.

This is the *only* architecture that works for a user with no home box, and it matches deployed practice (Delta Chat/Chatmail, Signal, Matrix/Sygnal — all wake-and-fetch with content-free push).

14.4 The always-on-box user

The Pi/NAS/VPS is the **durable authority**: it holds the mailbox, terminates client protocols (JMAP/IMAP, §8), and participates in the mesh. The owner's phone/laptop are thin clients that sync to it (§5.6 device cluster). Reachability (the box is usually NAT'd) comes from the relay layer (§14.5). Brief box downtime is covered by the same relay-mailbox/gateway short-queue buffer as §14.3; once the box returns and fetches, the durable copy lives on the box.

14.5 Relay & buffer scaling

Relays (reachability). Run **many small, independent, stateless relay nodes** — libp2p circuit-relay v2 needs zero coordination between relays, so the fleet scales by adding cheap nodes. Relays are a **reachability hop only**, with tight caps (go-libp2p defaults: ~128 reservations, 16 circuits/peer, 2 min / 128 KB per circuit). Discovery SHOULD use an **operator-run static/rendezvous list**, not sole reliance on the public DHT.

> **Honest limit (MUST disclose):** the public libp2p/IPFS DHT + relay path is designed for *brief hole-punch assistance*, not sustained mailbox sync — undialable-peer dial timeouts and republish load make it unsuitable as the sole production discovery/relay path. DMTAP deployments SHOULD run their own relay fleet with tuned caps and rendezvous discovery, using the DHT (if at all) only for opportunistic discovery. Direct connectivity (IPv6, hole-punch) is always preferred; relays carry the residual hard-NAT minority (§4.3).

Buffers (offline holding). Ciphertext for an offline node is held in a **content-blind relay-mailbox with a short TTL and delete-after-inactivity** (Chatmail model: ~20-day message TTL, ~90-day inactive-account purge as reference defaults). This decouples availability from any single peer's uptime while keeping per-account cost near-zero (no long-term archive).

Peer buffering (§4.3) is an alternative when no hosted buffer is desired, but its durability is tied to the buffering peer's uptime — weak for mobile-only. The gateway's short queue (§7.4) is only the legacy-translation hop and MUST NOT become a store.

> **Honest limit:** a relay-mailbox is a **buffer, not an archive** — a node offline past the TTL loses undelivered mail. Durability MUST land at the recipient's edge once fetched. Senders retry (§2.6) within their own deadline regardless.

14.6 Hosted multi-tenant topology (the operator)

For an operator hosting many mailboxes (Envoir Cloud, §12), the sensible horizontally-scalable shape is **stateless routed front + per-tenant sandboxed compute + object-storage persistence + scale-to-zero**:

- **Stateless front:** hostname-routed proxy on an **Anycast** network, TLS-terminating, backhauling to the tenant's cell.
- **Per-tenant isolation:** one logical app / sandbox (microVM, e.g. Firecracker) **per customer**, so a compromised tenant cannot read another's secrets or content; equivalently a namespace-per-tenant + sandboxed runtime on Kubernetes.
- **Storage:** per-account **encrypted object-storage bucket** (§ Envoir Cloud uses R2/Tigris, free egress); isolation enforced at the app/bucket boundary, not the platform sandbox alone.
- **Scale-to-zero + push-wake:** idle mailboxes cost only storage; a request/push cold-boots the cell (~sub-second). This fits mailboxes (idle most of the time) and the §14.3 push model.
- **Multi-region:** place each tenant's cell in its home region behind the Anycast front; control-plane metadata (tiny) in a shared control-plane DB (Neon), content in per-tenant buckets.

Because there is no long-term archive on the hot path and mailboxes are mostly idle, the per-mailbox cost floor is near-zero (a documented ~1 GB RAM / 1 CPU serves thousands of light mailboxes), which is what makes the operator economics in §12 work.

14.7 Grounding

Patterns grounded in: KumoMTA (shared-nothing MTA, egress pools), AWS SES (per-ISP managed warmup), SendGrid/Mailgun (per-stream pools, health-check demotion), RFC 5321 §5.1 (MX priority), Cloudflare (Anycast inbound), Delta Chat/Chatmail + Signal + Matrix/Sygnal (wake-and-fetch push, relay-mailbox), APNs/FCM (best-effort, 4 KB, throttling), libp2p circuit-relay v2 + Protocol Labs DHT findings (relay caps, discovery), Fly Machines ”one app per customer” + Kubernetes multi-tenancy (per-tenant sandbox, scale-to-zero). See §11 for the full bibliography.

15. Normative & Informative References

DMTAP is a **profile that composes existing standards**. This section is the authoritative reference list. **Normative** references are required to implement DMTAP correctly; **Informative** references are precedent, rationale, or optional interop. Full text is not reproduced here — RFCs are permanently available at `https://www.rfc-editor.org/rfc/rfcNNNN` and W3C/OpenID/other specs at the URLs given. DMTAP cites them by identifier and profiles them; where DMTAP narrows or extends a referenced spec, the DMTAP text is normative for DMTAP.

15.1 Normative — cryptography & encoding

Ref	Title	DMTAP use
RFC 9180	Hybrid Public Key Encryption (HPKE)	seal-to-key encryption; DHKEM(X25519)/HK
RFC 8032	EdDSA (Ed25519)	signatures (§1, §2)
RFC 7748	Elliptic Curves for Security (X25519)	key agreement (§1)
RFC 9420	The Messaging Layer Security (MLS) Protocol	group/session security for 1:1, groups, files, m
RFC 9750	The MLS Architecture	Delivery/Authentication Service roles → mesh
RFC 8949	Concise Binary Object Representation (CBOR)	wire serialization; **deterministic (core) enc**
FIPS 203	ML-KEM (Module-Lattice KEM)	PQ KEM (suite 0x02, §1.1)
FIPS 204	ML-DSA (Module-Lattice signatures)	PQ signatures (suite 0x02, §1.1)
BLAKE3	BLAKE3 cryptographic hash	content addressing (§2.2), hash-agile prefix; n

15.2 Normative — transport, mesh & naming

Ref	Title	DMTAP use
libp2p specs	Kademlia DHT, Circuit Relay v2, DCUtR, AutoNAT v2, Noise	mesh, reachability, N
RFC 9000	QUIC	primary transport; co
RFC 1035 / 9460	DNS / SVCB & HTTPS RRs	`name→key` records; un
RFC 6376	DomainKeys Identified Mail (DKIM)	gateway signs-as-dom
RFC 8461 / 7672	MTA-STS / DANE (SMTP TLS)	enforce TLS on the le

15.3 Normative — legacy mail interop (the gateway & client edges)

Ref	Title	DMTAP use
RFC 5321	Simple Mail Transfer Protocol (SMTP)	gateway legacy world; MX priority/
RFC 5322	Internet Message Format	RFC822 MOTE translation at the g
RFC 2045–2049	MIME	attachment/body representation acro
RFC 9051	IMAP4rev2	client-compat surface projected from
RFC 3501	IMAP4rev1	broader legacy-client compatibility (§
RFC 6409	Message Submission (port 587)	client outbound submission (§8)
RFC 1939	POP3	optional legacy-client compatibility (§
RFC 8620 / 8621	JMAP (Core / Mail)	native modern client sync surface (§8
RFC 4791	Calendaring Extensions to WebDAV (CalDAV)	legacy calendar-client compat, project
RFC 6352	vCard Extensions to WebDAV (CardDAV)	legacy contacts-client compat, project
RFC 5545	iCalendar	calendar object body format (§8.4)
RFC 6350	vCard 4.0	contact object body format (§8.4)
RFC 8984 / 9553	JSCalendar / JSContact	native JSON calendar/contact repres

15.4 Normative — identity, auth & anti-abuse

Ref	Title	DMTAP use
RFC 6749	OAuth 2.0 Authorization Framework	base framework for th
RFC 7636	PKCE	REQUIRED on any D
RFC 9449	OAuth 2.0 Demonstrating Proof of Possession (DPoP)	key-bound, sender-con
RFC 9068	JWT Profile for OAuth 2.0 Access Tokens	token format for the C
RFC 7009 / 7662	OAuth Token Revocation / Introspection	session revocation wit

Ref	Title	DMTAP use
RFC 8414	OAuth 2.0 Authorization Server Metadata	`.well-known` discovery
RFC 9700	Best Current Practice for OAuth 2.0 Security	security baseline for §
RFC 9576 / 9577 / 9578	Privacy Pass (Architecture / HTTP / Issuance)	anonymous anti-abuse
WebAuthn L2	Web Authentication (W3C Rec, L2; L3 CR)	origin-bound login cer
CTAP2	Client to Authenticator Protocol (FIDO2)	passkey/authenticator

15.5 Informative — precedent & rationale

Ref	Title
RFC 9635	Grant Negotiation and Authorization Protocol (GNA
draft-ietf-oauth-v2-1	OAuth 2.1
ERC-4361 / CAIP-122	Sign-In with Ethereum / Sign-In with X
W3C DID Core 1.0	Decentralized Identifiers
IndieAuth (living std)	Identity = a URL/domain you own
SIOP v2 (draft)	Self-Issued OpenID Provider
Solid-OIDC (CG report)	WebID + DPoP over OIDC
Nostr NIP-07 / NIP-98	keypair browser signing / HTTP auth
UCAN v1.0	User-Controlled Authorization Networks
Sphinx (Danezis & Goldberg, 2009)	mix packet format
Loopix (USENIX Sec 2017)	mixnet design
Anonymity Trilemma (IEEE S&P 2018)	anonymity latency/bandwidth
CONIKS (USENIX Sec 2015) / **RFC 6962**	key transparency / Certificate Transparency
SLIP-0039 / **RFC 9591 (FROST)**	Shamir mnemonic / threshold signatures
Chatmail / Delta Chat, Matrix/Sygnal, Signal	minimal mail server, push gateway, sealed sender
draft-connolly-cfrg-xwing-kem	X-Wing hybrid KEM
draft-ietf-mls-pq-ciphersuites / -combiner	PQ-MLS
draft-kohbrok-mls-dmls	Decentralized MLS
draft-yun-privacypass-arc	Anonymous Rate-Limited Credentials

15.6 On reproduction

DMTAP does not vendor the full text of referenced standards. RFCs are reproducible under BCP 78 / the IETF Trust Legal Provisions with their notices intact, but referencing by identifier is the norm and avoids staleness. Implementers should read the referenced specs directly at the URLs above; this document defines only the **DMTAP profile** over them.

16. Numeric Parameters (v0)

Interoperability requires agreed numbers, not qualitative ”hours”/”minutes.” This appendix fixes the v0 defaults. Each is a **parameter** an implementation MAY tune within stated bounds, but the defaults here are what a conformance run (§10) checks. All are versioned with the protocol.

16.1 Time, replay & clocks

Parameter	Default	Notes
Clock-skew tolerance	±120 s	max accepted difference between `ts` and receive
Challenge/nonce validity window	120 s	server-issued single-use nonce for auth (§13) & a
Replay cache retention	300 s	seen-`id` / seen-nonce cache MUST cover the ske
MOTE `expires` max	1 year	cap on requested client-side expiry
Sender retry deadline	72 h	after which an undelivered MOTE fails to the us
Sender retry backoff	exp, base 30 s, cap 1 h	exponential with jitter

Nodes MUST NOT rely on synchronized clocks for correctness — only for the skew-bounded windows above and for ordering hints.

16.2 Naming, KT & DHT

Parameter	Default	Notes
Key-name length	8 words (80 bits)	§3.9.1; 12 words (128 bits) for adversary-proof mode
Key-name wordlist size	1024	language-agnostic; +1 checksum word
KT signed-tree-head poll	6 h	client re-checks its own entry (self-monitoring)
KT gossip interval (v1)	1 h	tree-head gossip for equivocation detection
DHT location-record TTL	2 h	signed; republish before expiry (§4.2)
DHT republish interval	45 min	< TTL, with jitter
DHT lookup redundancy (K)	20	store at K closest; S/Kademlia disjoint paths 3
Location seq-number	monotonic u64	rollback defense; reject older-or-equal

16.3 Mixnet & privacy

Parameter	Default	Notes
Mix path length	3 hops	Sphinx onion (§4.4)
Per-hop mix delay	exp, mean 5 s	Poisson mixing; `private` tier
Cover-traffic rate	Poisson, mean 30 s/msg	tunable per node; higher = more privacy, more b
Padded packet size (bucket)	2 KiB	Sphinx constant-length cell after padding
`private`-tier end-to-end latency	seconds–minutes	consequence of mixing (§6)

16.4 File tiers & transfer

Parameter	Default	Notes
Inline attachment threshold	64 KiB	rides the message (§2.5)
Normal-file threshold	4 MiB (4 chunks)	chunks via mixnet — full privacy (§6.5)
Chunk size	1 MiB	fixed; content-addressed (§5.5)
Large-file bulk	> 4 MiB	fast/onion bulk path — weaker privacy (§6.5)
Swarm parallelism	8 sources	per-file concurrent fetch

16.5 Anti-abuse

Parameter	Default	Notes
Cold-sender PoW	memory-hard (Argon2id), adaptive	last-resort tier (§9.4)
PoW puzzle scope	`id ‖ recipient ‖ nonce(epoch)`	fresh epoch nonce to prevent precomput
Unknown-issuer token budget	0	self-issued/unvetted → no rate budget (
Requests-area retention	30 days	unproven cold MOTEs held here, not in
Postage redemption check	online (issuer)	no offline bearer acceptance (§9.5.1)

16.6 Transport & relay

Parameter	Default	Notes
Relay reservation TTL	1 h	libp2p circuit-relay v2 (§14.5)
Relay per-circuit cap	2 min / 128 KiB	brief hole-punch assist only — not sustained sync
Reachability ladder	IPv6 → hole-punch → relay	prefer direct (§4.3)
Offline-buffer TTL	20 days	relay-mailbox message hold (§14.5)
Inactive-account purge	90 days	relay-mailbox (§14.5)
Push payload	4 KiB, content-free	wake-and-fetch only (§14.3)

16.7 Crypto suites

`suite`	Sign	KEM	AEAD	Hash	S
`0x01`	Ed25519	X25519 (HPKE)	ChaCha20-Poly1305	BLAKE3-256	v
`0x02`	Ed25519+ML-DSA-65	X-Wing (X25519+ML-KEM-768)	ChaCha20-Poly1305	BLAKE3-256	P

All numeric values here are v0 defaults; a future protocol version MAY revise them, and capability negotiation (§10.2) carries any non-default profile.